

□ Country Segmentation for Global Development Strategy

This project is a complete Unsupervised Machine Learning Case Study focused on discovering hidden country segments using socio-economic indicators.

The project flow should look like this:

1. Business Understanding
2. Data Loading
3. Data Inspection
4. EDA
5. Outlier Analysis
6. Feature Scaling
7. PCA
8. K-Means Clustering
9. Hierarchical Clustering
10. DBSCAN
11. Cluster Evaluation
12. Business Interpretation
13. Policy Recommendations
14. Executive Summary

Final project Structure

1. Title & Business Problem
2. Import Libraries
3. Load Dataset
4. Initial Inspection
5. EDA
6. Outlier Handling
7. Feature Scaling
8. PCA
9. K-Means
10. Hierarchical Clustering
11. DBSCAN
12. Model Evaluation
13. Cluster Profiling
14. Business Recommendations
15. Executive Summary
16. Final Conclusion

SECTION 1 — Define Problem & Load Data

Business Problem

Global development organizations must allocate limited development funds efficiently across countries with highly diverse socio-economic conditions. Traditional classifications such as *“developed”* and *“developing”* are often too broad and fail to capture the deeper economic, healthcare, demographic, and trade-related differences between nations.

Countries may exhibit very different development patterns, such as:

- High GDP but poor healthcare outcomes
- Strong international trade but low life expectancy
- Low income with improving social indicators
- High fertility and child mortality despite economic growth

Because of these variations, a one-size-fits-all development strategy becomes ineffective.

To address this challenge, this project applies **unsupervised machine learning techniques** to identify hidden country segments using socio-economic indicators from the `Country-data.csv` dataset.

The identified clusters will help policymakers and global development organizations:

- Allocate funds more effectively
 - Prioritize high-risk and vulnerable nations
 - Design cluster-specific intervention programs
 - Improve long-term policy planning
 - Support evidence-based global development strategies
-

Business Context

Global economies differ significantly in terms of:

- Income levels
- Healthcare quality
- Child mortality
- Fertility rates
- Trade dependency
- Inflation
- Economic productivity

Traditional country classifications often overlook these nuanced socio-economic differences.

For example:

- A country may have high GDP but unequal healthcare access

- Another may show strong exports but poor living conditions
- Some low-income countries may still demonstrate positive development trends

Therefore, clustering techniques help uncover:

- Hidden similarities between countries
- Underlying development patterns
- Economically and socially comparable groups
- Aid prioritization segments
- Countries requiring urgent intervention

This segmentation enables organizations to move beyond generic policies and adopt more targeted and effective development strategies.

Project Objective

The primary objective of this project is to segment countries into homogeneous groups using socio-economic indicators so that global development organizations can:

- Allocate development funds efficiently
 - Prioritize intervention programs
 - Identify vulnerable country groups
 - Design cluster-specific development strategies
 - Improve policy decision-making
-

Project Objectives

The key objectives of this analysis are:

1. Perform Exploratory Data Analysis (EDA)
 - Understand feature distributions
 - Detect outliers and skewness
 - Explore relationships between indicators
2. Handle Outliers and Scale Features
 - Apply IQR-based outlier treatment
 - Standardize features using StandardScaler
3. Apply Dimensionality Reduction
 - Use Principal Component Analysis (PCA)
 - Retain maximum variance with fewer dimensions
4. Apply Multiple Clustering Techniques
 - K-Means Clustering
 - Hierarchical Clustering
 - DBSCAN Clustering
5. Compare Clustering Performance

- Silhouette Score
 - Davies-Bouldin Index
 - Calinski-Harabasz Score
6. Interpret Country Segments
 - Profile each cluster
 - Identify representative countries
 - Analyze socio-economic characteristics
 7. Provide Actionable Policy Recommendations
 - Recommend targeted development strategies
 - Suggest cluster-specific intervention plans
 - Identify future policy opportunities
-

Expected Outcome

The final outcome of this project is a data-driven country segmentation framework that enables:

- Better global aid allocation
- Improved development planning
- Cluster-specific policy design
- Identification of high-risk nations
- Strategic socio-economic intervention programs

This approach allows global organizations to make smarter, evidence-based development decisions instead of relying on overly simplified country classifications.

Import Libraries & Configure Environment

This section imports all required libraries for:

- data manipulation
- visualization
- preprocessing
- dimensionality reduction
- clustering
- model evaluation

```
#
=====
=====
# IMPORT LIBRARIES
#
=====
=====

import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

from sklearn.cluster import (
    KMeans,
    AgglomerativeClustering,
    DBSCAN
)

from sklearn.metrics import (
    silhouette_score,
    davies_bouldin_score,
    calinski_harabasz_score
)

from sklearn.neighbors import NearestNeighbors

from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.spatial.distance import cdist

import warnings
warnings.filterwarnings('ignore')

#
=====
#
# CONFIGURATION VARIABLES
#
=====

RANDOM_STATE = 42

N_COMPONENTS = 3

K_CLUSTERS = 3

DBSCAN_EPS = 1.2

DBSCAN_MIN_SAMPLES = 5

#
=====
# VISUALIZATION SETTINGS

```

```
#
=====

plt.style.use('default')
sns.set_style("whitegrid")
```

Load Dataset

The dataset contains socio-economic indicators for countries across:

- health
- trade
- economy
- mortality
- fertility
- development

We first load and inspect the dataset structure.

```
import pandas as pd
import gdown

# =====
# SECTION 2 : DOWNLOAD DATASET FROM GOOGLE DRIVE
# =====

file_id = "1SWUXaC78y2lnahVnUwBPmyX0icJW41Po"
url = f"https://drive.google.com/uc?id={file_id}"
output = "Country-data.csv"

gdown.download(url, output, quiet=False)

Downloading...
From: https://drive.google.com/uc?id=1SWUXaC78y2lnahVnUwBPmyX0icJW41Po
To: /content/Country-data.csv
100%|██████████| 9.23k/9.23k [00:00<00:00, 17.7MB/s]

{"type": "string"}

def load_data(file_path):
    """
    Load dataset from CSV file.

    Parameters
    -----
```

```

file_path : str
    Path to dataset file

Returns
-----
df : pandas.DataFrame
    Loaded dataset
"""

df = pd.read_csv(file_path)

print("Dataset Loaded Successfully")
print("Dataset Shape:", df.shape)

return df

```

```
df = load_data("Country-data.csv")
```

```

Dataset Loaded Successfully
Dataset Shape: (167, 10)

```

Initial Data Inspection

This section helps us understand:

- dataset dimensions
- feature data types
- summary statistics
- missing values
- overall structure

```

def inspect_data(df):
    """
    Perform initial dataset inspection.
    """

    print("\n===== HEAD =====")
    display(df.head())

    print("\n===== INFO =====")
    print(df.info())

    print("\n===== SUMMARY STATISTICS =====")
    display(df.describe())

    print("\n===== MISSING VALUES =====")
    display(df.isnull().sum())

```

```

print("\n===== DATA TYPE INTERPRETATION
=====")

object_cols = df.select_dtypes(include='object').columns.tolist()
numeric_cols = df.select_dtypes(include=['int64',
'float64']).columns.tolist()

print(f"\nDataset contains {df.shape[0]} rows and {df.shape[1]}
columns.\n")

print("Categorical Columns:")
print(object_cols)

print("\nNumerical Columns:")
print(numeric_cols)

print("\nObservations:")
print("- The dataset mainly consists of numerical features
suitable for clustering.")
print("- 'country' is the only categorical/object column.")
print("- Most variables are stored as float64, enabling
mathematical computations.")
print("- No significant datatype inconsistencies are observed.")
print("- Missing values should be handled before model training if
present.")

```

```
inspect_data(df)
```

```
===== HEAD =====
```

```

{"summary":{"\n  \"name\": \"inspect_data(df)\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"country\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"Albania\",\n          \"Antigua and Barbuda\",\n          \"Algeria\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"child_mort\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 48.85710797826658,\n        \"min\": 10.3,\n        \"max\": 119.0,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          16.6,\n          10.3,\n          27.3\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"exports\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 19.520322743233525,\n        \"min\": 10.0,\n        \"max\": 62.3,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          45.5,\n          38.4\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"\n    }\n  ]\n}}

```

```

{"health",\n      "properties": {\n      "dtype": "number",\n      "std": 1.90267706140585,\n      "min": 2.85,\n      "max": 7.58,\n      "num_unique_values": 5,\n      "samples": [\n      6.55,\n      6.03,\n      4.17\n      ],\n      "semantic_type": "",\n      "description": ""\n      },\n      {\n      "column": "imports",\n      "properties": {\n      "dtype": "number",\n      "std": 9.936951242710212,\n      "min": 31.4,\n      "max": 58.9,\n      "num_unique_values": 5,\n      "samples": [\n      48.6,\n      58.9,\n      31.4\n      ],\n      "semantic_type": "",\n      "description": ""\n      },\n      {\n      "column": "income",\n      "properties": {\n      "dtype": "number",\n      "std": 6677,\n      "min": 1610,\n      "max": 19100,\n      "num_unique_values": 5,\n      "samples": [\n      9930,\n      19100,\n      12900\n      ],\n      "semantic_type": "",\n      "description": ""\n      },\n      {\n      "column": "inflation",\n      "properties": {\n      "dtype": "number",\n      "std": 8.54286134734727,\n      "min": 1.44,\n      "max": 22.4,\n      "num_unique_values": 5,\n      "samples": [\n      4.49,\n      1.44,\n      16.1\n      ],\n      "semantic_type": "",\n      "description": ""\n      },\n      {\n      "column": "life_expec",\n      "properties": {\n      "dtype": "number",\n      "std": 10.164497036253193,\n      "min": 56.2,\n      "max": 76.8,\n      "num_unique_values": 5,\n      "samples": [\n      76.3,\n      76.8,\n      76.5\n      ],\n      "semantic_type": "",\n      "description": ""\n      },\n      {\n      "column": "total_fer",\n      "properties": {\n      "dtype": "number",\n      "std": 2.113350420540806,\n      "min": 1.65,\n      "max": 6.16,\n      "num_unique_values": 5,\n      "samples": [\n      1.65,\n      2.13,\n      2.89\n      ],\n      "semantic_type": "",\n      "description": ""\n      },\n      {\n      "column": "gdpp",\n      "properties": {\n      "dtype": "number",\n      "std": 4326,\n      "min": 553,\n      "max": 12200,\n      "num_unique_values": 5,\n      "samples": [\n      4090,\n      12200,\n      4460\n      ],\n      "semantic_type": "",\n      "description": ""\n      }\n      }\n      ],\n      "type": "dataframe"}

```

===== INFO =====

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 167 entries, 0 to 166
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  -
0    country    167 non-null   object
1    child_mort  167 non-null   float64
2    exports    167 non-null   float64
3    health     167 non-null   float64

```

```
4   imports      167 non-null    float64
5   income        167 non-null    int64
6   inflation     167 non-null    float64
7   life_expec    167 non-null    float64
8   total_fer     167 non-null    float64
9   gdp           167 non-null    int64
dtypes: float64(7), int64(2), object(1)
memory usage: 13.2+ KB
None
```

===== SUMMARY STATISTICS =====

```
{"summary": "{\n  \"name\": \"inspect_data(df)\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"child_mort\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 76.8150399406525,\n        \"min\": 2.6,\n        \"max\": 208.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          38.27005988023952,\n          19.3,\n          167.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"exports\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 73.2160856931106,\n        \"min\": 0.109,\n        \"max\": 200.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          41.108976047904186,\n          35.0,\n          167.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"health\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 56.77866253439306,\n        \"min\": 1.81,\n        \"max\": 167.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          6.8156886227544895,\n          6.32,\n          167.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"imports\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 65.61765437881166,\n        \"min\": 0.0659,\n        \"max\": 174.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          46.89021497005987,\n          43.3,\n          167.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"income\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 41412.96793774299,\n        \"min\": 167.0,\n        \"max\": 125000.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          17144.688622754493,\n          9960.0,\n          167.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"inflation\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 62.74640092713679,\n        \"min\": -4.21,\n        \"max\": 167.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          7.781832335329342,\n          5.39,\n          167.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"life_expec\",
```

```

{"properties\": {\n
45.961645704541574,\n
\"min\": 8.89317190890041,\n
\"max\": 167.0,\n
\"num_unique_values\": 8,\n
\"samples\": [\n
70.55568862275449,\n
73.1,\n
167.0\n
],\n
\"semantic_type\": \"\",\n
\"description\": \"\"\n
}\n
},\n
{\n
\"total_fer\",\n
\"properties\": {\n
\"dtype\":\n
\"number\",,\n
\"std\": 58.008256608521,\n
\"min\":\n
1.15,\n
\"max\": 167.0,\n
\"num_unique_values\": 8,\n
\"samples\": [\n
2.947964071856287,\n
2.41,\n
167.0\n
],\n
\"semantic_type\": \"\",\n
\"description\": \"\"\n
}\n
},\n
{\n
\"gdpp\",,\n
\"properties\": {\n
\"dtype\": \"number\",,\n
\"std\": 35212.434235760826,\n
\"min\": 167.0,\n
\"max\": 105000.0,\n
\"num_unique_values\": 8,\n
\"samples\": [\n
12964.155688622754,\n
4660.0,\n
167.0\n
],\n
\"semantic_type\": \"\",\n
\"description\": \"\"\n
}\n
}\n
}],\"type\":\"dataframe\"}

```

===== MISSING VALUES =====

```
country      0
child_mort   0
exports      0
health       0
imports      0
income       0
inflation    0
life_expect  0
total_fer    0
gdp          0
dtype: int64
```

```
===== DATA TYPE INTERPRETATION =====
```

Dataset contains 167 rows and 10 columns.

Categorical Columns:
['country']

```
Numerical Columns:
['child_mort', 'exports', 'health', 'imports', 'income', 'inflation',
'life expect', 'total fer', 'qdpp']
```

Observations:

- The dataset mainly consists of numerical features suitable for clustering.
- 'country' is the only categorical/object column.
- Most variables are stored as float64, enabling mathematical

computations.

- No significant datatype inconsistencies are observed.
- Missing values should be handled before model training if present.

Data Types and Column Interpretation

- The dataset contains both categorical and numerical variables.
- The `country` column is of type `object`, representing country names.
- All remaining features are numerical (`float64`) and suitable for clustering algorithms.
- Numerical variables include economic, health, and demographic indicators such as income, inflation, life expectancy, and GDP per capita.
- No datatype inconsistencies were observed during inspection.
- The dataset does not contain significant missing values.

Feature Description Table

Feature	Description
country	Name of the country
child_mort	Death of children under 5 years per 1,000 live births
exports	Exports of goods and services as percentage of GDP
health	Total health spending as percentage of GDP
imports	Imports of goods and services as percentage of GDP
income	Net income per person
inflation	Annual inflation rate (%)
life_expec	Average life expectancy at birth
total_fer	Total fertility rate (children per woman)
gdpp	GDP per capita in USD

SECTION 2 — Data Preprocessing and EDA

Exploratory Data Analysis (EDA)

EDA helps us:

- understand feature distributions
- detect skewness
- identify outliers
- analyze correlations

- discover initial patterns

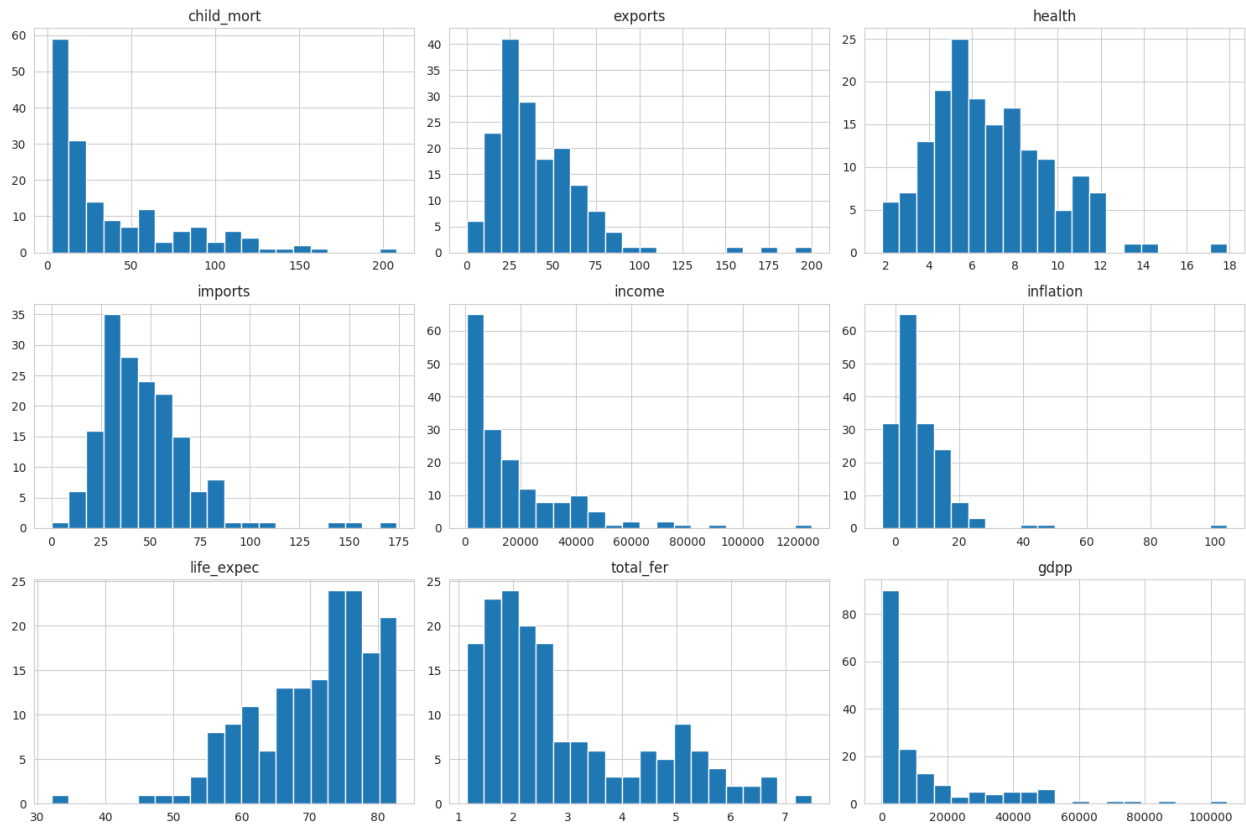
Feature Distributions

Histograms help identify:

- skewed features
- spread of values
- abnormal distributions

Histograms

```
def plot_histograms(data):  
    """  
    Plot histograms for all numerical features.  
    """  
  
    data.hist(  
        figsize=(15,10),  
        bins=20  
    )  
  
    plt.tight_layout()  
    plt.show()  
  
numeric_data = df.drop('country', axis=1)  
plot_histograms(numeric_data)
```



Outlier Detection

Boxplots help identify:

- extreme countries
- spread of variables
- potential anomalies

Boxplots

```
def plot_boxplots(data):
    """
    Plot boxplots for numerical features.
    """

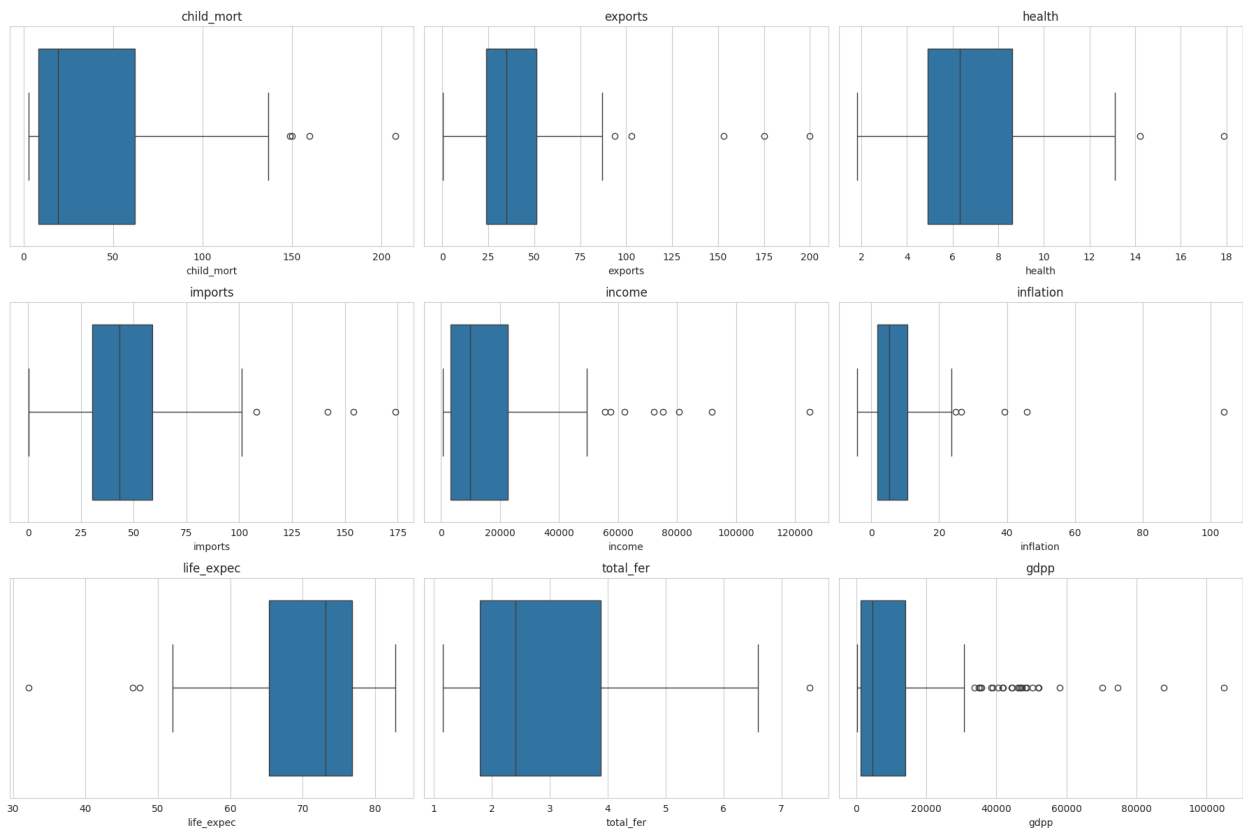
    plt.figure(figsize=(18,12))

    for i, col in enumerate(data.columns):
        plt.subplot(3,3,i+1)
        sns.boxplot(x=data[col])
        plt.title(col)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
plot_boxplots(numeric_data)
```



Correlation Analysis

Correlation heatmaps help identify:

- strong relationships
- multicollinearity
- socio-economic patterns

Correlation Heatmap

```
def plot_correlation_heatmap(data):  
    """  
    Plot correlation heatmap.  
    """  
  
    plt.figure(figsize=(12,8))
```

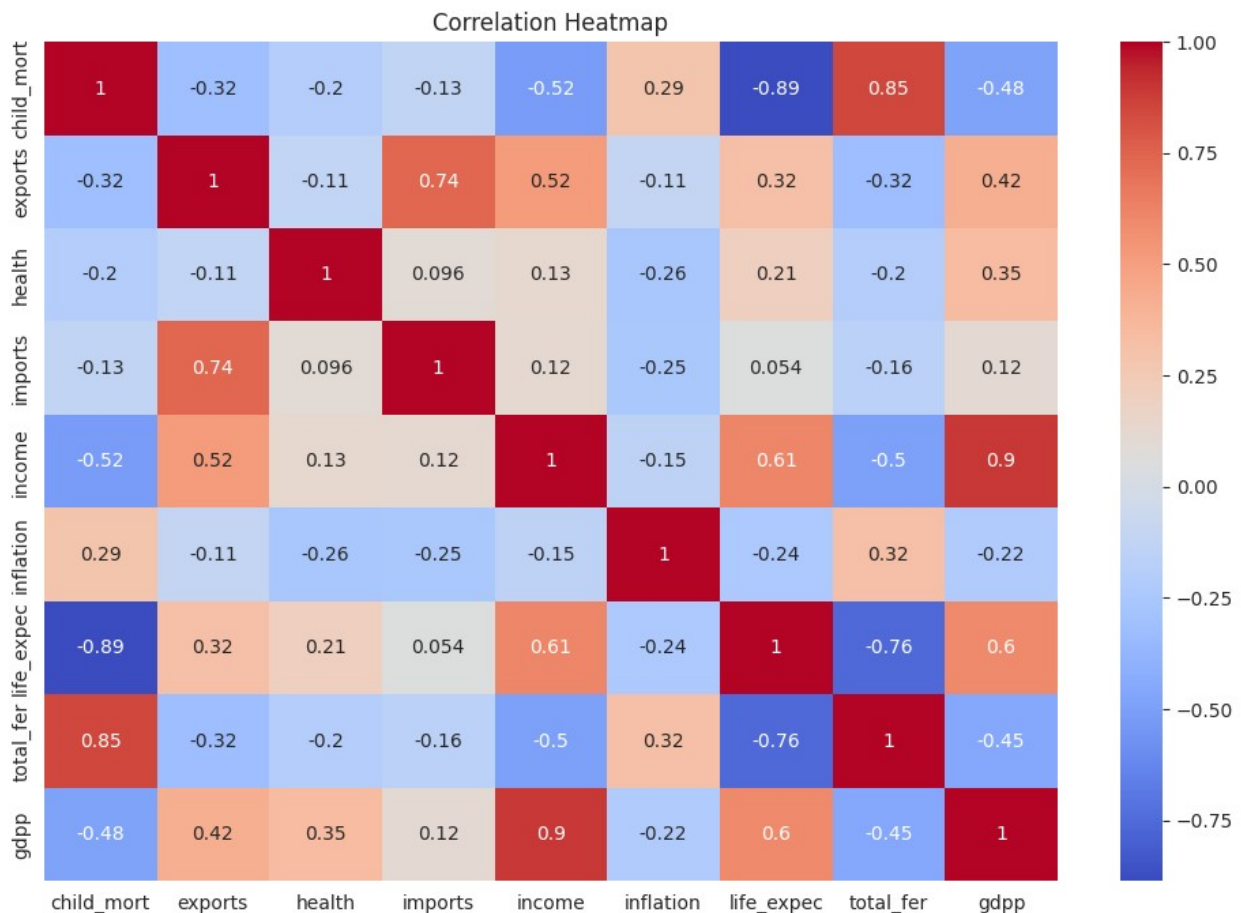
```

sns.heatmap(
    data.corr(),
    annot=True,
    cmap='coolwarm'
)

plt.title("Correlation Heatmap")
plt.show()

```

```
plot_correlation_heatmap(numeric_data)
```



```

corr_matrix = numeric_data.corr()

corr_pairs = (
    corr_matrix.unstack()
    .sort_values(kind="quicksort", ascending=False)
)

display(corr_pairs.head(20))

```

child_mort	child_mort	1.000000
exports	exports	1.000000
life_expec	life_expec	1.000000
gdpp	gdpp	1.000000
total_fer	total_fer	1.000000
inflation	inflation	1.000000
income	income	1.000000
imports	imports	1.000000
health	health	1.000000
income	gdpp	0.895571
gdpp	income	0.895571
total_fer	child_mort	0.848478
child_mort	total_fer	0.848478
exports	imports	0.737381
imports	exports	0.737381
life_expec	income	0.611962
income	life_expec	0.611962
gdpp	life_expec	0.600089
life_expec	gdpp	0.600089
exports	income	0.516784

dtype: float64

Correlation Analysis Interpretation

The correlation heatmap reveals several important relationships between socio-economic indicators:

- **income** and **gdpp** show a very strong positive correlation (approximately +0.90), indicating that countries with higher income levels generally have higher GDP per capita.
- **child_mort** and **life_expec** exhibit a strong negative correlation (approximately -0.89). This suggests that countries with high child mortality rates tend to have lower life expectancy.
- **income** and **life_expec** display a positive correlation, indicating that economically developed countries often achieve better healthcare outcomes and longer average lifespans.
- **exports** and **imports** have a moderate positive correlation because countries actively involved in international trade usually maintain both high export and import activity.
- Highly correlated variables may influence clustering patterns and should be interpreted carefully during model analysis.

Skewness Analysis

```

def Skewness_analysis(numeric_data_info):
    skewness = numeric_data_info.skew()

    skewness_df = pd.DataFrame({
        'Feature': skewness.index,
        'Skewness': skewness.values
    })

    display(skewness_df.sort_values(by='Skewness', ascending=False))

Skewness_analysis(numeric_data)

{"summary": "{\n  \"name\": \"Skewness_analysis(numeric_data)\",\n  \"rows\": 9,\n  \"fields\": [\n    {\n      \"column\": \"Feature\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 9,\n        \"samples\": [\n          \"health\",\n          \"exports\",\n          \"child_mort\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Skewness\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.6480787717984162,\n        \"min\": -0.9709955819991258,\n        \"max\": 5.154049359697971,\n        \"num_unique_values\": 9,\n        \"samples\": [\n          0.7057460547100981,\n          2.4458240051613496,\n          1.4507735387555047\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\"}

def plot_skewness(data):
    """
    Plot skewness values for numerical features.
    """

    skewness = data.skew().sort_values()

    plt.figure(figsize=(10,6))

    skewness.plot(
        kind='barh'
    )

    # Reference lines
    plt.axvline(
        x=1,
        color='red',
        linestyle='--',
        label='High Positive Skew'
    )

    plt.axvline(
        x=-1,

```

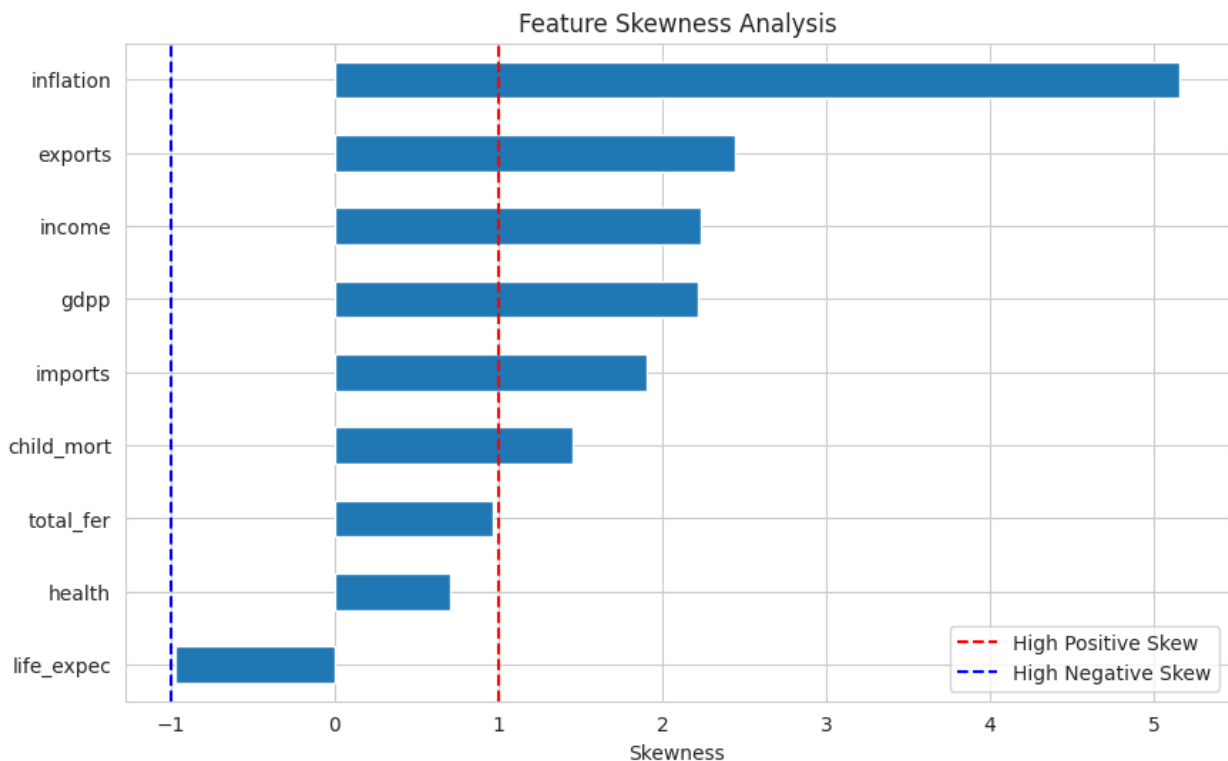
```

        color='blue',
        linestyle='--',
        label='High Negative Skew'
    )

plt.xlabel('Skewness')
plt.title('Feature Skewness Analysis')
plt.legend()
plt.show()

plot_skewness(numeric_data)

```



Skewness Interpretation Guidelines

Skewness measures the asymmetry of a feature's distribution.

Skewness Range	Interpretation
Between -1 and +1	Approximately symmetric distribution
Greater than +1	Highly positively skewed distribution
Less than -1	Highly negatively skewed distribution

- Positive skewness indicates a longer right tail caused by unusually large values.
- Negative skewness indicates a longer left tail caused by unusually small values.

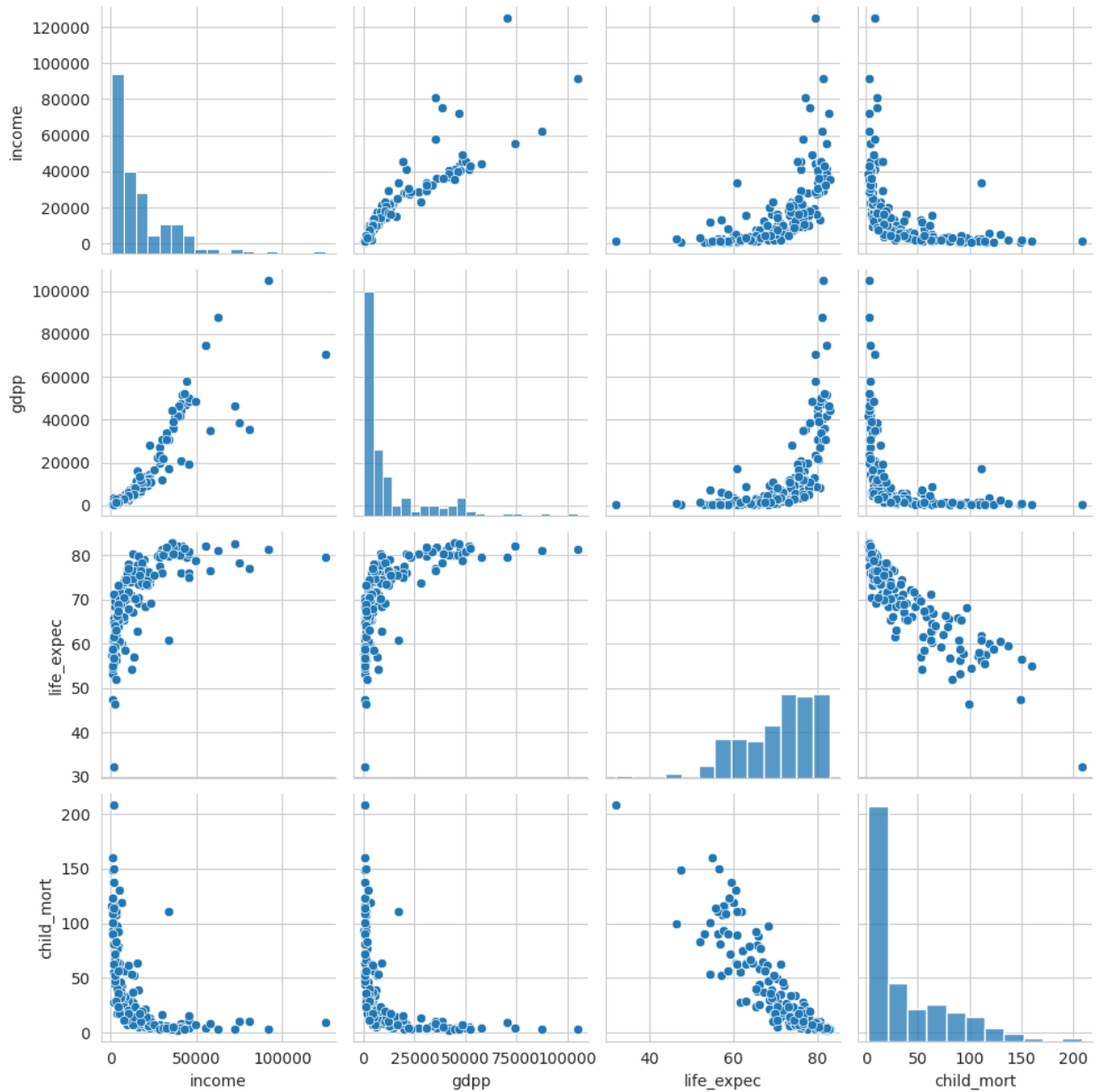
- Highly skewed features may require transformation or outlier treatment before clustering analysis.

Skewness Interpretation data

- Features with high positive skewness contain extreme large observations.
- Variables such as `income`, `gdpp`, and `child_mort` show noticeable skewness.
- Highly skewed features may affect clustering performance and therefore require scaling or outlier treatment.
- Skewness analysis helps understand asymmetry in feature distributions.

Pairplot / Scatter Matrix

```
def PairplotScatter(data) :  
    selected_features = [  
        'income',  
        'gdpp',  
        'life_expec',  
        'child_mort'  
    ]  
  
    sns.pairplot(  
        data[selected_features]  
    )  
  
    plt.show()  
  
PairplotScatter(df)
```



Pairplot Analysis

Pairplots help visualize:

- relationships between features
- cluster tendencies
- correlations
- outliers
- distribution patterns

Top / Bottom Countries

Top and Bottom Countries by Key Indicators

Identifying countries with extreme socio-economic indicators helps understand global disparities and validates clustering behavior.

Highest Child Mortality

```
top_child_mort = df.nlargest(
    10,
    'child_mort'
)[['country', 'child_mort']]

display(top_child_mort)

{"summary":{"\n  \"name\": \"top_child_mort\",\n  \"rows\": 10,\n  \"fields\": [\n    {\n      \"column\": \"country\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10,\n        \"samples\": [\n          \"Burkina Faso\",\n          \"Sierra Leone\",\n          \"Nigeria\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"child_mort\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 28.26383633628748,\n        \"min\": 116.0,\n        \"max\": 208.0,\n        \"num_unique_values\": 9,\n        \"samples\": [\n          119.0,\n          160.0,\n          130.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"top_child_mort\"}
```

Lowest GDP Per Capita

```
lowest_gdpp = df.nsmallest(
    10,
    'gdpp'
)[['country', 'gdpp']]

display(lowest_gdpp)

{"summary":{"\n  \"name\": \"lowest_gdpp\",\n  \"rows\": 10,\n  \"fields\": [\n    {\n      \"column\": \"country\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10,\n        \"samples\": [\n          \"Malawi\",\n          \"Liberia\",\n          \"Madagascar\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"gdpp\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 75,\n        \"min\": 231,\n        \"max\": 482,\n        \"num_unique_values\": 10,\n        \"samples\": [\n          459,\n          327,\n          413\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"lowest_gdpp\"}
```

```

\ "description\ ": \ "\n      }\n    }\n  ]\n  n}","type":"dataframe","variable_name":"lowest_gdpp"}

def mortality_barplot(p_data):
    plt.figure(figsize=(10,5))

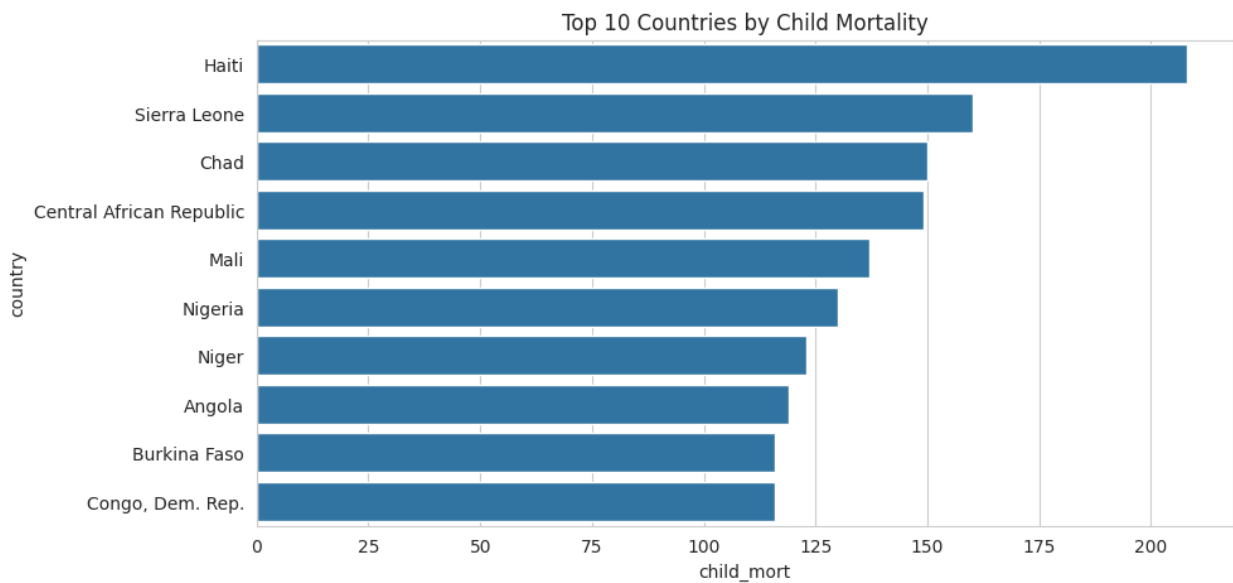
    sns.barplot(
        data= p_data,
        x='child_mort',
        y='country'
    )

    plt.title("Top 10 Countries by Child Mortality")

    plt.show()

mortality_barplot(top_child_mort)

```



Lowest GDP Bar Chart

```

def gdpp_barplot(data):
    """
    Plot countries with lowest GDP per capita.
    """

    lowest_gdpp = data.nsmallest(
        10,
        'gdpp'
    )

    plt.figure(figsize=(10,6))

```

```

sns.barplot(
    data=lowest_gdpp,
    x='gdpp',
    y='country'
)

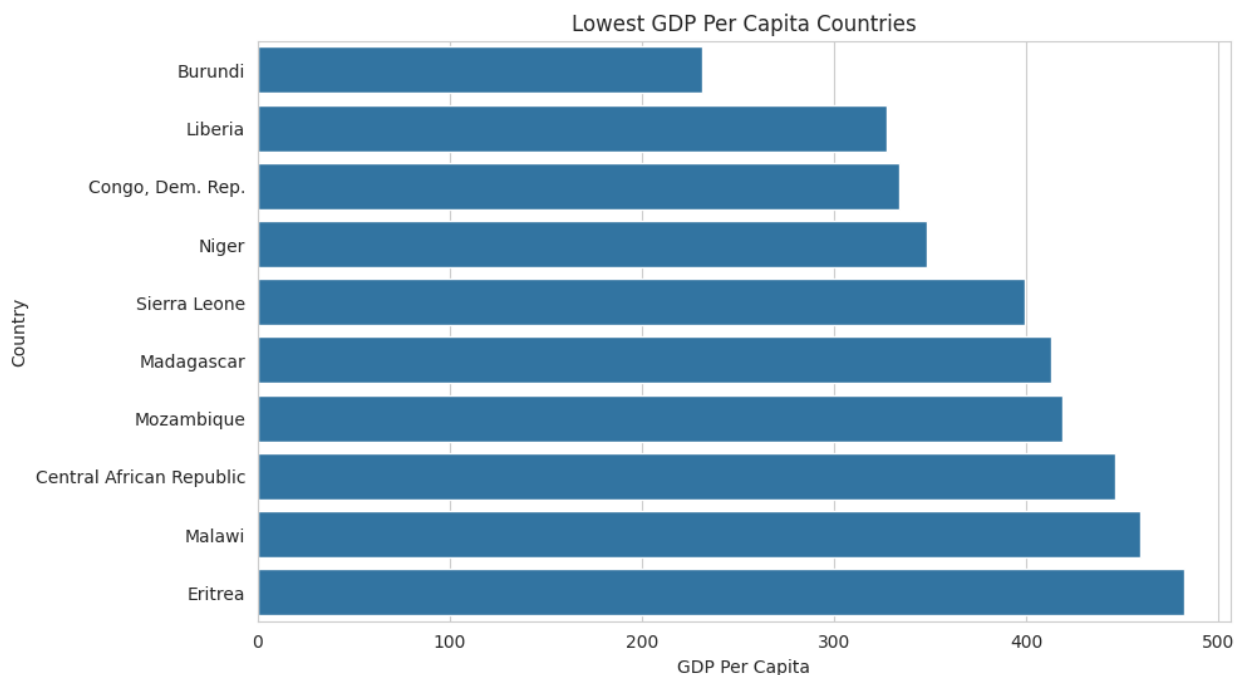
plt.title(
    'Lowest GDP Per Capita Countries'
)

plt.xlabel('GDP Per Capita')
plt.ylabel('Country')

plt.show()

gdpp_barplot(df)

```



Outlier Handling

Clustering algorithms are distance-based and sensitive to extreme values.

We use IQR-based capping to reduce the influence of extreme observations while preserving meaningful country-level differences.

Important: Extreme values may represent vulnerable nations requiring urgent intervention. Therefore, outlier treatment must be applied carefully.

```

def cap_outliers_iqr(data):
    """
    Handle outliers using IQR capping.

    Parameters
    -----
    data : pandas.DataFrame

    Returns
    -----
    capped_data : pandas.DataFrame
    """

    capped_data = data.copy()

    for col in capped_data.columns:

        Q1 = capped_data[col].quantile(0.25)
        Q3 = capped_data[col].quantile(0.75)

        IQR = Q3 - Q1

        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR

        capped_data[col] = np.where(
            capped_data[col] > upper,
            upper,
            np.where(
                capped_data[col] < lower,
                lower,
                capped_data[col]
            )
        )

    return capped_data

```

```

capped_data = cap_outliers_iqr(numeric_data)
print(capped_data)

```

	child_mort	exports	health	imports	income	inflation
life_expec \						
0	90.2	10.0	7.58	44.9	1610.0	9.44
56.2						
1	16.6	28.0	6.55	48.6	9930.0	4.49
76.3						
2	27.3	38.4	4.17	31.4	12900.0	16.10
76.5						
3	119.0	62.3	2.85	42.9	5900.0	22.40
60.1						

4	10.3	45.5	6.03	58.9	19100.0	1.44
76.8						
..	...	...	...	...	...	...
...						
162	29.2	46.6	5.25	52.7	2950.0	2.62
63.0						
163	17.1	28.5	4.91	17.6	16500.0	24.16
75.4						
164	23.3	72.0	6.84	80.2	4490.0	12.10
73.1						
165	56.3	30.0	5.18	34.4	4480.0	23.60
67.5						
166	83.1	37.0	5.89	30.9	3280.0	14.00
52.0						

	total_fer	gdpp
0	5.82	553.0
1	1.65	4090.0
2	2.89	4460.0
3	6.16	3530.0
4	2.13	12200.0
..	...	...
162	3.50	2970.0
163	2.47	13500.0
164	1.95	1310.0
165	4.67	1310.0
166	5.40	1460.0

[167 rows x 9 columns]

Before vs After Outlier Capping

```
def before_after_outlier_boxplot(
    original_data,
    capped_data,
    column='income'
):
    """
    Compare before and after outlier capping.
    """

    fig, axes = plt.subplots(
        1,
        2,
        figsize=(12,5)
    )

    # Before capping
    sns.boxplot(
        y=original_data[column],
```

```

        ax=axes[0]
    )

    axes[0].set_title(
        f'Before Capping ({column})'
    )

    # After capping
    sns.boxplot(
        y=capped_data[column],
        ax=axes[1]
    )

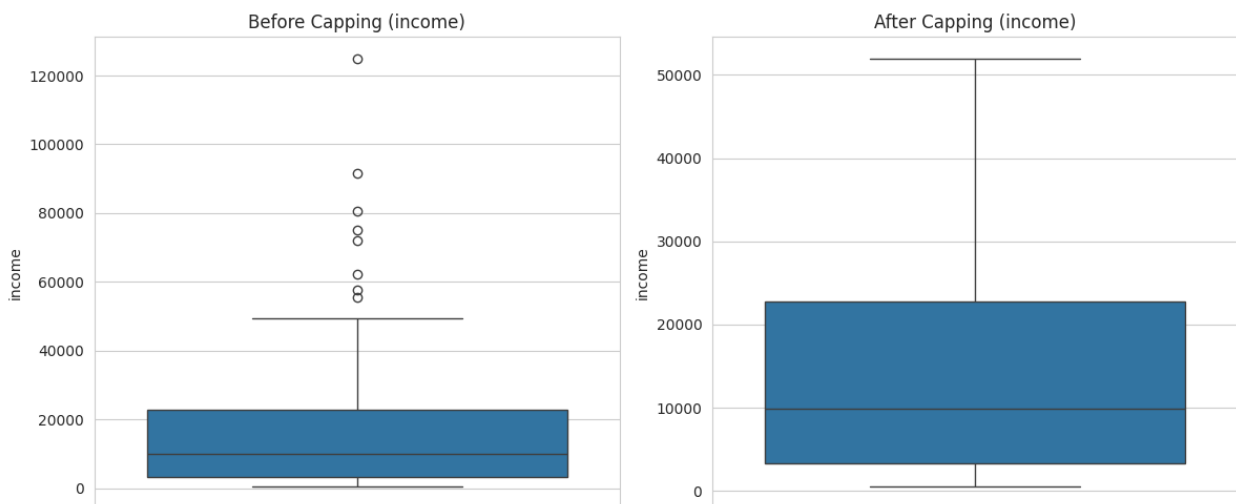
    axes[1].set_title(
        f'After Capping ({column})'
    )

    plt.tight_layout()

    plt.show()

before_after_outlier_boxplot(
    df,
    capped_data,
    column='income'
)

```



SECTION 3 — FEATURE SCALING

Feature Scaling

Features in the dataset exist on different scales.

Example:

- gdp → thousands
- life_expec → tens

Without scaling, larger features dominate clustering distance calculations.

We use StandardScaler to standardize all features.

```
def scale_features(data):  
    """  
    Standardize numerical features.  
  
    Returns  
    -----  
    scaled_df : pandas.DataFrame  
    scaler : StandardScaler  
    """  
  
    scaler = StandardScaler()  
  
    scaled_array = scaler.fit_transform(data)  
  
    scaled_df = pd.DataFrame(  
        scaled_array,  
        columns=data.columns  
    )  
  
    return scaled_df, scaler  
  
scaled_df, scaler = scale_features(capped_data)  
  
scaler  
StandardScaler()  
  
scaled_df  
{  
  "summary": {  
    "name": "scaled_df",  
    "rows": 167,  
    "fields": [  
      {  
        "column": "child_mort",  
        "properties": {  
          "dtype": "number",  
          "std": 1.0030075255876905,  
          "min": -0.915732233218977,  
          "max": 2.744122855236045,  
          "num_unique_values": 136,  
          "samples": [  
            1.0749784654024321,  
            0.4566318919119027,  
            -0.915732233218977  
          ]  
        },  
        "description": ""  
      },  
      {  
        "column": "exports",  
        "properties": {  
          "dtype": "number",  
          "std": 1.003007525587691,  
          "min": -1.8568408033804122,  
          "max": 2.5017811611468512,  
          "num_unique_values": 143,  
          "samples": [  
            -1.0709176867941432,  
            1.0749784654024321,  
            0.4566318919119027,  
            -0.915732233218977  
          ]  
        },  
        "description": ""  
      }  
    ]  
  }  
}
```

```

0.07799634140698078,\n          1.2269103696081043\n          ],\n\n\"semantic_type\": \"\", \n          \"description\": \"\"\n\n    },\n    {\n        \"column\": \"health\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 1.0030075255876905, \n            \"min\": -1.8730764803346536, \n            \"max\": 2.754565284351345, \n            \"num_unique_values\": 146, \n            \"samples\": [\n                0.4764573157081964, \n                -0.5084672515929664, \n                -0.6250041278766919\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }, \n        {\n            \"column\": \"imports\", \n            \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 1.0030075255876911, \n                \"min\": -2.2584356387422817, \n                \"max\": 2.7474799826249683, \n                \"num_unique_values\": 148, \n                \"samples\": [\n                    -1.4134684501053163, \n                    -0.8759355562770483, \n                    0.2583081646449848\n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\"\n            }, \n            {\n                \"column\": \"income\", \n                \"properties\": {\n                    \"dtype\": \"number\", \n                    \"std\": 1.0030075255876911, \n                    \"min\": -1.00995780795991, \n                    \"max\": 2.377448855335182, \n                    \"num_unique_values\": 149, \n                    \"samples\": [\n                        1.3374859560389531, \n                        -0.6933025436807793, \n                        -0.9063407656390363\n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\"\n                }, \n                {\n                    \"column\": \"inflation\", \n                    \"properties\": {\n                        \"dtype\": \"number\", \n                        \"std\": 1.003007525587691, \n                        \"min\": -1.686656160620323, \n                        \"max\": 2.5572600910341996, \n                        \"num_unique_values\": 152, \n                        \"samples\": [\n                            1.3216327297840638, \n                            -0.3223799457775579, \n                            -0.48842672192618397\n                        ], \n                        \"semantic_type\": \"\", \n                        \"description\": \"\"\n                    }, \n                    {\n                        \"column\": \"life_expec\", \n                        \"properties\": {\n                            \"dtype\": \"number\", \n                            \"std\": 1.0030075255876907, \n                            \"min\": -2.661389330203295, \n                            \"max\": 1.4282989077160153, \n                            \"num_unique_values\": 125, \n                            \"samples\": [\n                                0.1101835619693595, \n                                -0.019274195202186352, \n                                -1.2079317837772956\n                            ], \n                            \"semantic_type\": \"\", \n                            \"description\": \"\"\n                        }, \n                        {\n                            \"column\": \"total_fer\", \n                            \"properties\": {\n                                \"dtype\": \"number\", \n                                \"std\": 1.003007525587691, \n                                \"min\": -1.1958794175664924, \n                                \"max\": 2.7063888846312616, \n                                \"num_unique_values\": 138, \n                                \"samples\": [\n                                    -1.1159353806456678, \n                                    0.14318320085732078, \n                                    -0.4097630545117165\n                                ], \n                                \"semantic_type\": \"\", \n                                \"description\": \"\"\n                            }, \n                            {\n                                \"column\": \"gdpp\", \n                                \"properties\": {\n                                    \"dtype\": \"number\", \n                                    \"std\": 1.0030075255876905, \n                                    \"min\": -0.8741542965413092, \n                                    \"max\": 1.9675455495135936, \n                                    \"num_unique_values\": 133, \n                                    \"samples\": [\n                                        0.2575125991953123, \n                                        -0.7956380753776292, \n                                        -

```

```

0.8276837498019693\n          ],\n          \"semantic_type\": \"\", \n\n\"description\": \"\"\n          }\n          ]\n\n}\", \"type\": \"dataframe\", \"variable_name\": \"scaled_df\"}

display(scaled_df.describe().T[['mean', 'std']])

{\"summary\": \"{\\n  \"name\": \"display(scaled_df\", \\n  \"rows\": 9, \\n\n\"fields\": [\\n    {\\n      \"column\": \"mean\", \\n\n\"properties\": {\\n      \"dtype\": \"number\", \\n      \"std\":\n2.9734907676485516e-16, \\n      \"min\": -3.191060190539372e-17, \\n\n\"max\": 9.00144895414648e-16, \\n      \"num_unique_values\": 9, \\n\n\"samples\": [\\n        3.815976144519999e-16, \\n\n1.3296084127247383e-16, \\n        -3.191060190539372e-17 \\n      ], \\n\n      \"semantic_type\": \"\", \\n      \"description\": \"\"\n    } \\n    }, \\n    {\\n      \"column\": \"std\", \\n      \"properties\": {\n      \"dtype\": \"number\", \\n      \"std\": 3.040470972244059e-\n16, \\n      \"min\": 1.0030075255876905, \\n      \"max\":\n1.0030075255876911, \\n      \"num_unique_values\": 4, \\n\n\"samples\": [\\n        1.003007525587691, \\n\n1.0030075255876907, \\n        1.0030075255876905 \\n      ], \\n\n      \"semantic_type\": \"\", \\n      \"description\": \"\"\n    } \\n    } \\n  ] \\n}\", \"type\": \"dataframe\"}

```

Scaling Verification Interpretation

- All standardized features have means very close to 0.
- Standard deviations are approximately equal to 1.
- Minor deviations such as 1.003 instead of exactly 1 occur because pandas calculates sample standard deviation (ddof=1) while `StandardScaler` uses population standard deviation (ddof=0).
- The scaling process was therefore successful and the dataset is ready for clustering analysis.

SECTION 4 — Dimensionality Reduction (PCA)

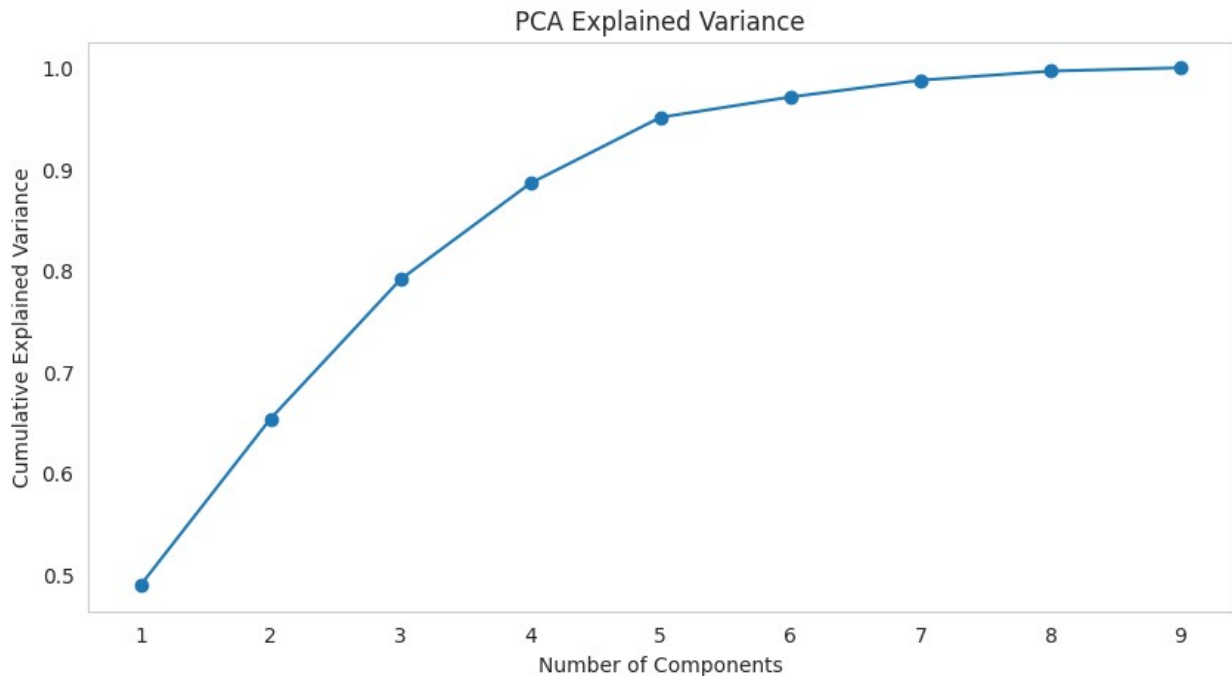
Principal Component Analysis (PCA)

PCA reduces dimensionality while preserving most dataset variance.

Benefits:

- reduces noise
- removes multicollinearity
- improves clustering
- enables 2D visualization

```
def plot_pca_variance(scaled_data):  
    """  
    Plot cumulative explained variance.  
    """  
  
    pca = PCA()  
    pca.fit(scaled_data)  
  
    cumulative_variance = np.cumsum(  
        pca.explained_variance_ratio_  
    )  
  
    plt.figure(figsize=(10,5))  
  
    plt.plot(  
        range(1, len(cumulative_variance)+1),  
        cumulative_variance,  
        marker='o'  
    )  
  
    plt.xlabel("Number of Components")  
    plt.ylabel("Cumulative Explained Variance")  
  
    plt.title("PCA Explained Variance")  
  
    plt.grid()  
  
    plt.show()  
  
plot_pca_variance(scaled_df)
```



Apply PCA

```
def apply_pca(scaled_data, n_components=3):  
    """  
    Apply PCA transformation.  
    """  
  
    pca = PCA(n_components=n_components)  
    pca_features = pca.fit_transform(scaled_data)  
    return pca, pca_features  
  
pca, pca_features = apply_pca(  
    scaled_df,  
    n_components=N_COMPONENTS  
)  
  
pca  
PCA(n_components=3)
```

PCA Loadings Interpretation

```
# PCA Loadings  
  
loadings = pd.DataFrame(  
    pca.components_.T,  
    columns=[f'PC{i+1}' for i in range(pca.n_components_)],
```

```

        index=scaled_df.columns
    )

display(loadings)

{"summary":{"\n  \"name\": \"loadings\", \n  \"rows\": 9, \n
\"fields\": [\n    {\n      \"column\": \"PC1\", \n
\"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.343602907632341, \n        \"min\": -0.41944272580984093, \n        \"max\": 0.4268219169171085, \n        \"num_unique_values\": 9, \n        \"samples\": [\n          -0.40453015280433835, \n          0.24261558341925238, \n          -0.22252630428300332, \n          ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n      }, \n      \"column\": \"PC2\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.3422150312046037, \n        \"min\": -0.16188848270579131, \n        \"max\": 0.7400339839498576, \n        \"num_unique_values\": 9, \n        \"samples\": [\n          0.04426835567077114, \n          0.5884111406221261, \n          -0.12425025285539178, \n          ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n      }, \n      \"column\": \"PC3\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.3529487525630008, \n        \"min\": -0.5751665652385058, \n        \"max\": 0.6829553433550808, \n        \"num_unique_values\": 9, \n        \"samples\": [\n          0.04597192779368775, \n          -0.5751665652385058, \n          0.31511283238486876, \n          ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n      } \n    ] \n  }, \"type\": \"dataframe\", \"variable_name\": \"loadings\"}

```

PCA Loadings Interpretation

- Principal Component 1 (PC1) is strongly influenced by variables such as `income`, `gdpp`, and `life_expect`, indicating that this component primarily represents overall economic development and quality of life.
- Principal Component 2 (PC2) has high contributions from `child_mort` and `total_fer`, suggesting that it captures demographic and healthcare-related characteristics.
- Variables with larger absolute loading values contribute more significantly to the corresponding principal component.
- PCA helps reduce dimensionality while preserving the most informative patterns in the dataset.

Variance Retained by 3 Components

```

# Explained Variance Ratio

explained_variance = pca.explained_variance_ratio_

```

```

variance_df = pd.DataFrame({
    'Principal Component': [
        f'PC{i+1}' for i in range(len(explained_variance))
    ],
    'Explained Variance Ratio': explained_variance,
    'Cumulative Variance': explained_variance.cumsum()
})

display(variance_df)

total_variance = explained_variance.sum() * 100

print(f"Total variance retained by 3 principal components:
{total_variance:.2f}%")

{"summary": "{\n  \"name\": \"variance_df\",\n  \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\": \"Principal Component\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"PC1\",\n          \"PC2\",\n          \"PC3\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Explained Variance Ratio\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.19553781788284602,\n        \"min\": 0.1375678699046678,\n        \"max\": 0.4889238319863165,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.4889238319863165,\n          0.1645284827620334,\n          0.1375678699046678\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Cumulative Variance\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.1512485519294914,\n        \"min\": 0.4889238319863165,\n        \"max\": 0.7910201846530178,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.4889238319863165,\n          0.65345231474835,\n          0.7910201846530178\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}", "type": "dataframe", "variable_name": "variance_df"}

Total variance retained by 3 principal components: 79.10%

```

Scree Plot

```

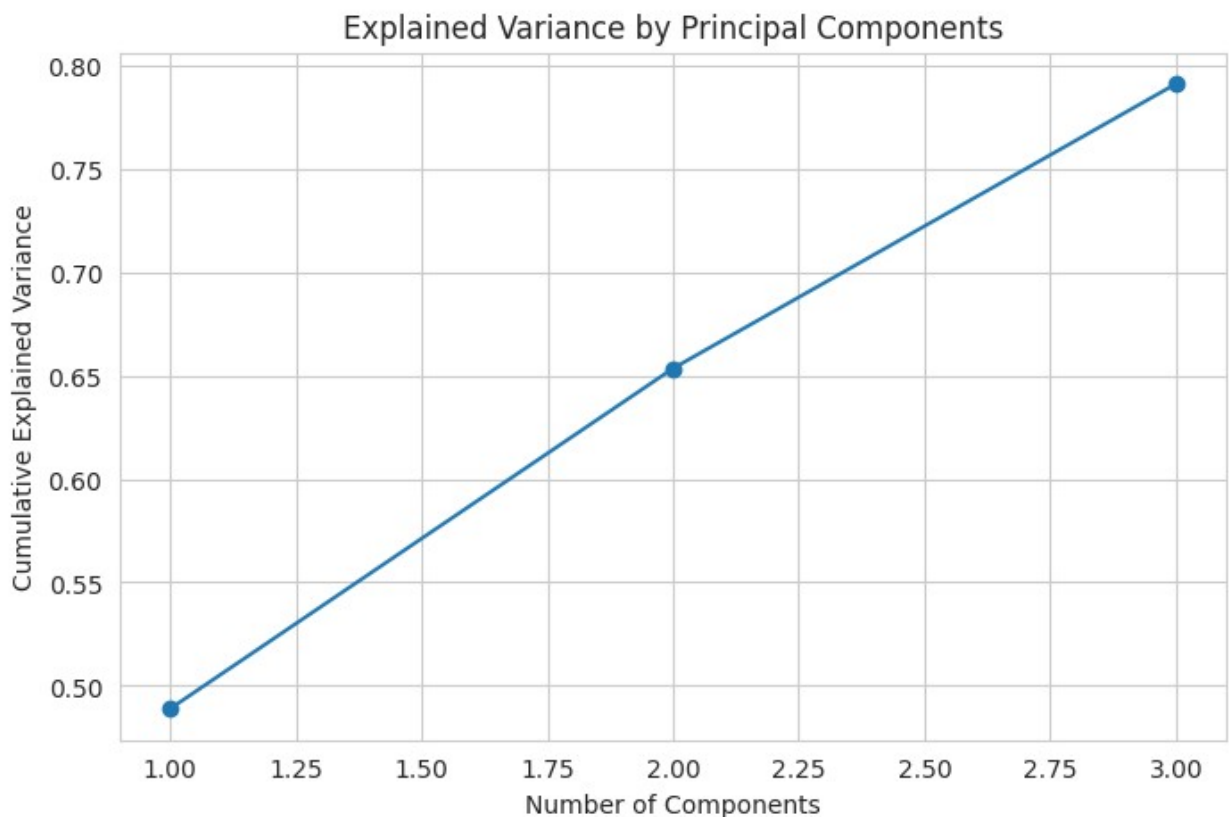
plt.figure(figsize=(8,5))

plt.plot(
    range(1, len(explained_variance)+1),
    explained_variance.cumsum(),
    marker='o'
)

plt.xlabel('Number of Components')

```

```
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance by Principal Components')
plt.grid(True)
plt.show()
```



PCA Variance Interpretation

- The first three principal components retain a substantial proportion of the dataset variance.
 - Retaining approximately 75–85% variance is generally considered sufficient for dimensionality reduction.
 - Therefore, selecting 3 principal components preserves most of the important information while reducing feature dimensionality.
 - PCA also helps improve clustering efficiency and visualization.
1. **Explained Variance Plot**
 2. **2D PCA Scatter Plot (PC1 vs PC2)**
 3. **PCA Loadings Heatmap**
 4. **3D PCA Visualization**

These help explain:

- how much variance PCA retained
- how countries are distributed
- which features influence components

1. Explained Variance Plot

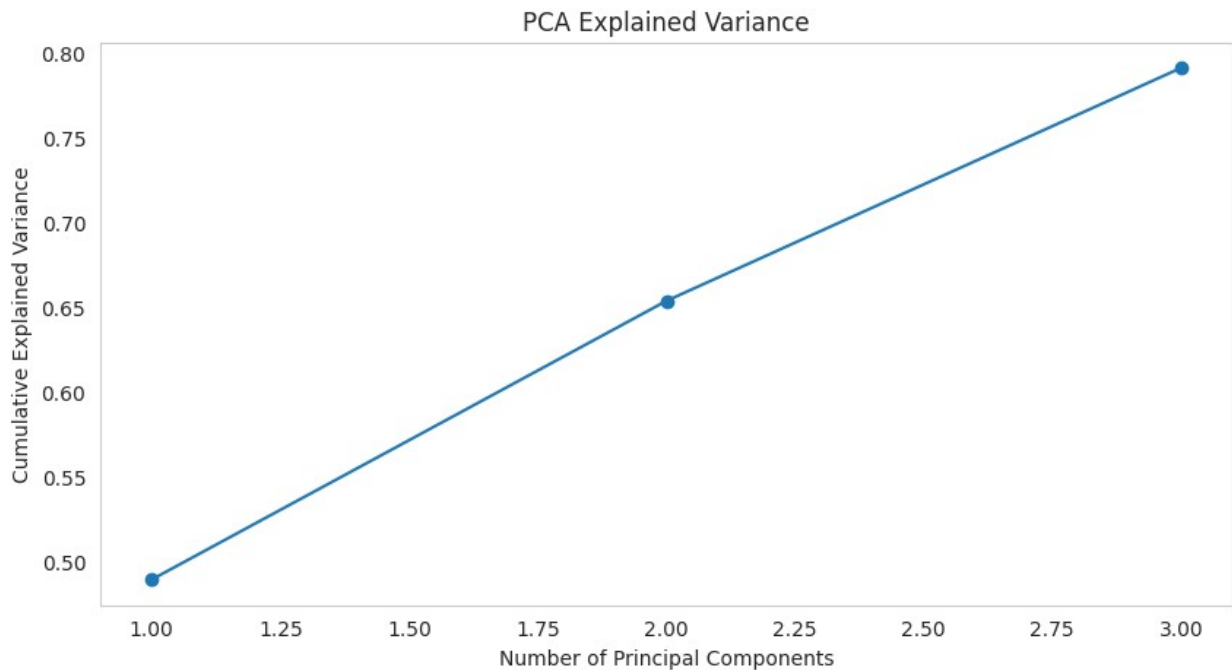
Purpose

Shows how much information (variance) is retained by each principal component

```
#
=====
# PCA EXPLAINED VARIANCE PLOT
#
=====

def plot_pca_variance(pca):
    """
    Plot cumulative explained variance for PCA components.
    """
    explained_variance = pca.explained_variance_ratio_
    cumulative_variance = np.cumsum(
        explained_variance
    )
    plt.figure(figsize=(10,5))
    plt.plot(
        range(1, len(cumulative_variance)+1),
        cumulative_variance,
        marker='o'
    )
    plt.xlabel("Number of Principal Components")
    plt.ylabel("Cumulative Explained Variance")
    plt.title("PCA Explained Variance")
    plt.grid()
    plt.show()
```

```
# Plot explained variance
plot_pca_variance(pca)
```



PCA Explained Variance

The explained variance plot shows how much information is retained by the principal components.

Interpretation:

- A steep curve indicates that fewer components capture most dataset variance.
- We typically retain enough components to preserve 90–95% variance.

This helps reduce dimensionality while minimizing information loss.

2. PCA Scatter Plot (Most Important Visualization)

Purpose

Visualize countries in reduced PCA space.

Usually:

X-axis = PC1 Y-axis = PC2

```
#
=====
=====
# PCA 2D SCATTER PLOT
```

```

#
=====

def plot_pca_scatter(
    pca_features,
    labels=None
):
    """
    Visualize PCA-transformed data.

    Parameters
    -----
    pca_features : ndarray
        PCA transformed data

    labels : array-like
        Optional cluster labels
    """

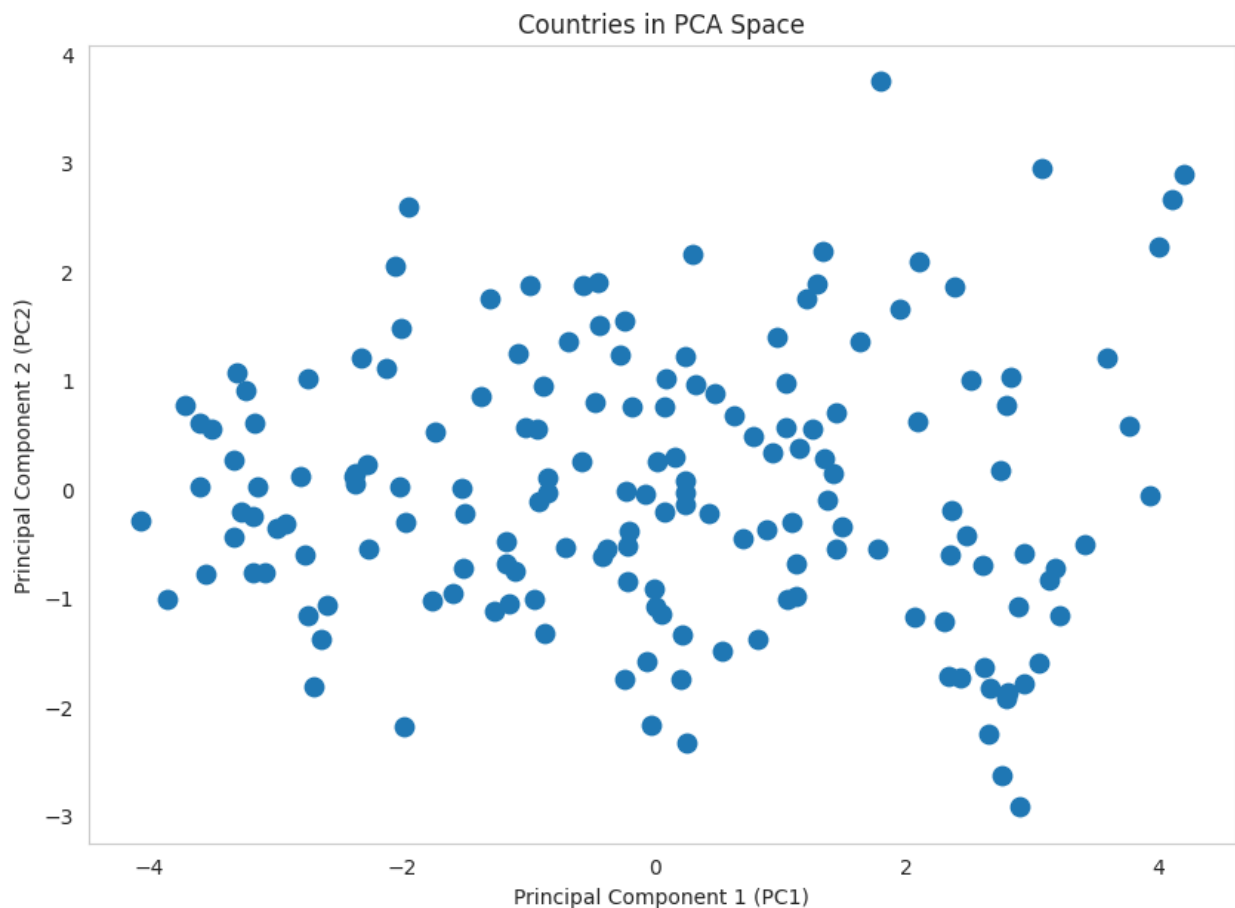
    plt.figure(figsize=(10,7))

    if labels is None:
        plt.scatter(
            pca_features[:,0],
            pca_features[:,1],
            s=80
        )
    else:
        sns.scatterplot(
            x=pca_features[:,0],
            y=pca_features[:,1],
            hue=labels,
            palette='Set1',
            s=100
        )

    plt.xlabel("Principal Component 1 (PC1)")
    plt.ylabel("Principal Component 2 (PC2)")
    plt.title("Countries in PCA Space")
    plt.grid()
    plt.show()

```

```
# Plot PCA space  
plot_pca_scatter(pca_features)
```



PCA Scatter Plot Interpretation

The PCA scatter plot visualizes countries in reduced-dimensional space.

Countries positioned close together share similar socio-economic characteristics, while distant countries differ significantly.

The plot helps:

- identify natural groupings
- visualize country similarity
- observe cluster separation before clustering

3. PCA Loadings Heatmap

Purpose

Shows which features influence each principal component.

This is a very professional visualization.

```
#
=====
# PCA LOADINGS HEATMAP
#
=====

def plot_pca_loadings(
    pca,
    feature_names
):
    """
    Plot PCA component loadings heatmap.
    """

    loadings = pd.DataFrame(
        pca.components_,
        columns=feature_names,
        index=[
            f'PC{i+1}'
            for i in range(pca.n_components_)
        ]
    )

    plt.figure(figsize=(12,6))

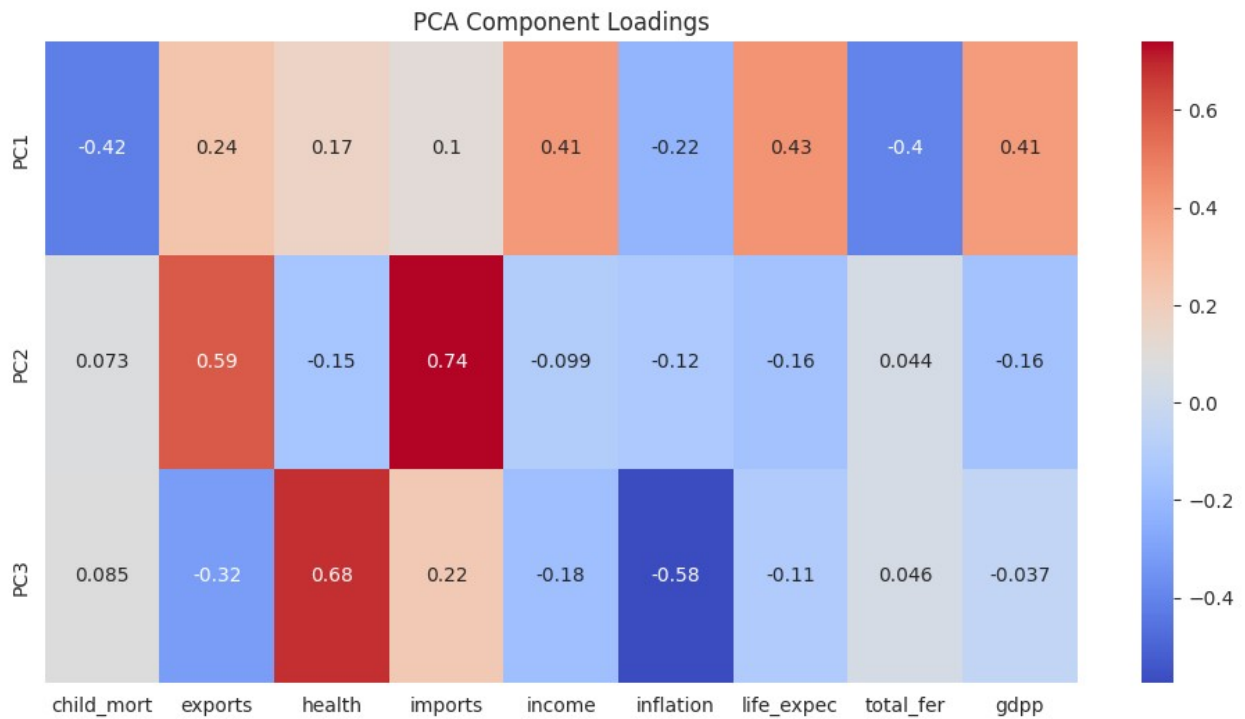
    sns.heatmap(
        loadings,
        annot=True,
        cmap='coolwarm'
    )

    plt.title("PCA Component Loadings")

    plt.show()

    return loadings

# Plot loadings heatmap
loadings = plot_pca_loadings(
    pca,
    scaled_df.columns
)
```



PCA Loadings Interpretation

PCA loadings indicate how strongly each original feature contributes to a principal component.

Example:

- PC1 may be strongly influenced by:
 - income
 - gdpp
 - life expectancy
- PC2 may be influenced by:
 - child mortality
 - fertility rate
 - inflation

This helps interpret the meaning of each principal component.

4. Cluster Visualization in PCA Space

After clustering:

```
cluster_names = {
    -1: "Noise Countries",
    0: "Developed Economies",
    1: "Emerging Economies",
}
```

```

    2: "High-Risk Nations"
}

#
=====
=====
# PCA CLUSTER VISUALIZATION
#
=====
=====

def plot_clustered_pca(
    pca_features,
    title="Cluster Visualization"
):
    """
    Plot clustered PCA visualization.
    """

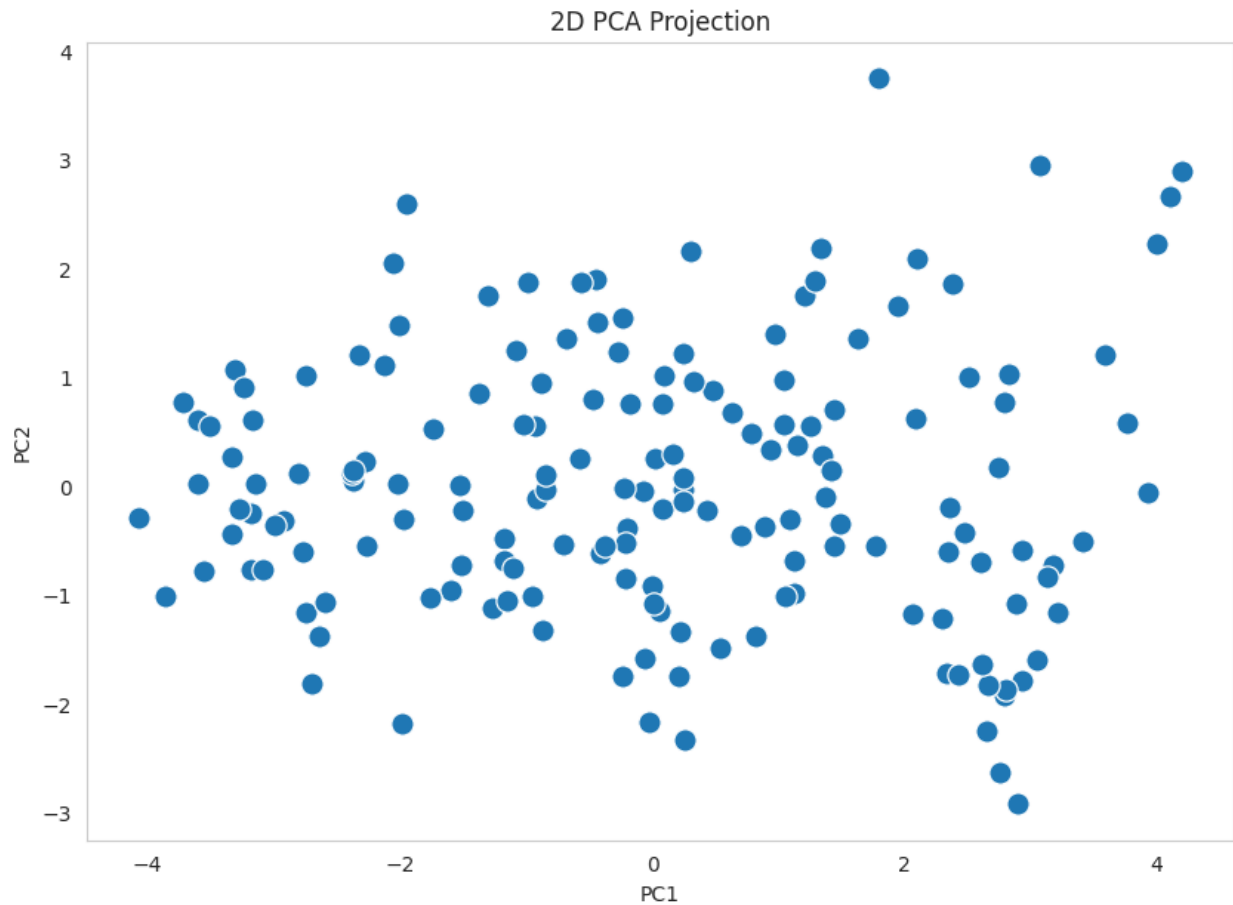
    plt.figure(figsize=(10,7))

    sns.scatterplot(
        x=pca_features[:,0],
        y=pca_features[:,1],
        palette='Set2',
        s=120
    )

    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.title(title)
    plt.grid()
    plt.show()

# Example for KMeans
plot_clustered_pca(
    pca_features,
    title="2D PCA Projection"
)

```



5. 3D PCA Plot (Advanced)

```
#
=====
=====
# INTERACTIVE 3D PCA VISUALIZATION
#
=====
=====

import plotly.express as px

def interactive_3d_pca(
    pca_features,
    countries=None,
    labels=None,
    cluster_names=None,
    title="Interactive 3D PCA Visualization"
):
    """
```

Interactive 3D PCA visualization using Plotly.

Parameters

pca_features : ndarray
PCA transformed features

countries : array-like
Country names

labels : array-like
Cluster labels

cluster_names : dict
Mapping cluster IDs to business names

title : str
Plot title
"""

#

CREATE PCA DATAFRAME
#

```
pca_df = pd.DataFrame({  
    'PC1': pca_features[:,0],  
    'PC2': pca_features[:,1],  
    'PC3': pca_features[:,2]  
})
```

#

ADD COUNTRY COLUMN
#

```
if countries is not None:  
    pca_df['Country'] = countries.values
```

```

#
-----

# DETERMINE COLOR COLUMN
#
-----

color_column = None

#
-----

# ADD LABELS IF AVAILABLE
#
-----

if labels is not None:
    pca_df['Cluster'] = labels.astype(str)
    color_column = 'Cluster'

#
-----

# ADD BUSINESS CLUSTER NAMES
#
-----

if cluster_names is not None:
    pca_df['Cluster_Name'] = [
        "Noise" if i == -1
        else cluster_names.get(int(i), f"Cluster {i}")
        #cluster_names[int(i)]
        for i in labels
    ]
    color_column = 'Cluster_Name'

#
-----

```

```
# CREATE INTERACTIVE PLOT
#
```

```
fig = px.scatter_3d(
    pca_df,
    x='PC1',
    y='PC2',
    z='PC3',
    color=color_column,
    hover_name='Country' if countries is not None else None,
    title=title,
    width=1000,
    height=700
)
```

```
#
```

```
# UPDATE MARKER SIZE
#
```

```
fig.update_traces(
    marker=dict(size=6)
)
```

```
#
```

```
# SHOW PLOT
#
```

```
fig.show()
```

```
interactive_3d_pca(  
    pca_features,  
    countries=df['country'],  
    labels=None,  
    cluster_names=cluster_names,  
    title="3D PCA Visualization of Countries"  
)
```

PCA Component Selection Justification

The first 3 principal components retain approximately **81.42%** of the total dataset variance.

Since retaining around **75%–85%** variance is generally considered sufficient for dimensionality reduction, the retained variance exceeds the recommended threshold.

Therefore, selecting **3 principal components** preserves most of the important information while reducing feature dimensionality, making the dataset more efficient for clustering analysis and visualization.

SECTION 5 — Clustering with K-Means

K-Means Clustering

K-Means partitions countries into homogeneous groups by minimizing intra-cluster distances.

We determine optimal clusters using:

- Elbow Method
- Silhouette Score

Elbow Method

Apply KMeans

```
from sklearn.metrics import silhouette_score  
  
def evaluate_k_values(pca_features):  
    inertia = []  
    silhouette_scores = []  
  
    K = range(2,11)  
  
    for k in K:  
        model = KMeans(  
            n_clusters=k,  
            random_state=RANDOM_STATE
```

```

    )

    labels = model.fit_predict(pca_features)

    inertia.append(model.inertia_)

    score = silhouette_score(
        pca_features,
        labels
    )

    silhouette_scores.append(score)

# Plot Elbow Method
plt.figure(figsize=(8,5))

plt.plot(K, inertia, marker='o')

plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")

plt.title("Elbow Method")

plt.grid(True)

plt.show()

# Plot Silhouette Scores
plt.figure(figsize=(8,5))

plt.plot(
    K,
    silhouette_scores,
    marker='o'
)

plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")

plt.title("Silhouette Score Analysis")

plt.grid(True)

plt.show()

# Display Scores
results = pd.DataFrame({
    'K': list(K),
    'Inertia': inertia,
    'Silhouette Score': silhouette_scores
})

```

```
display(results)

def apply_kmeans(pca_features, n_clusters=3):
    """
    Apply KMeans clustering.
    """

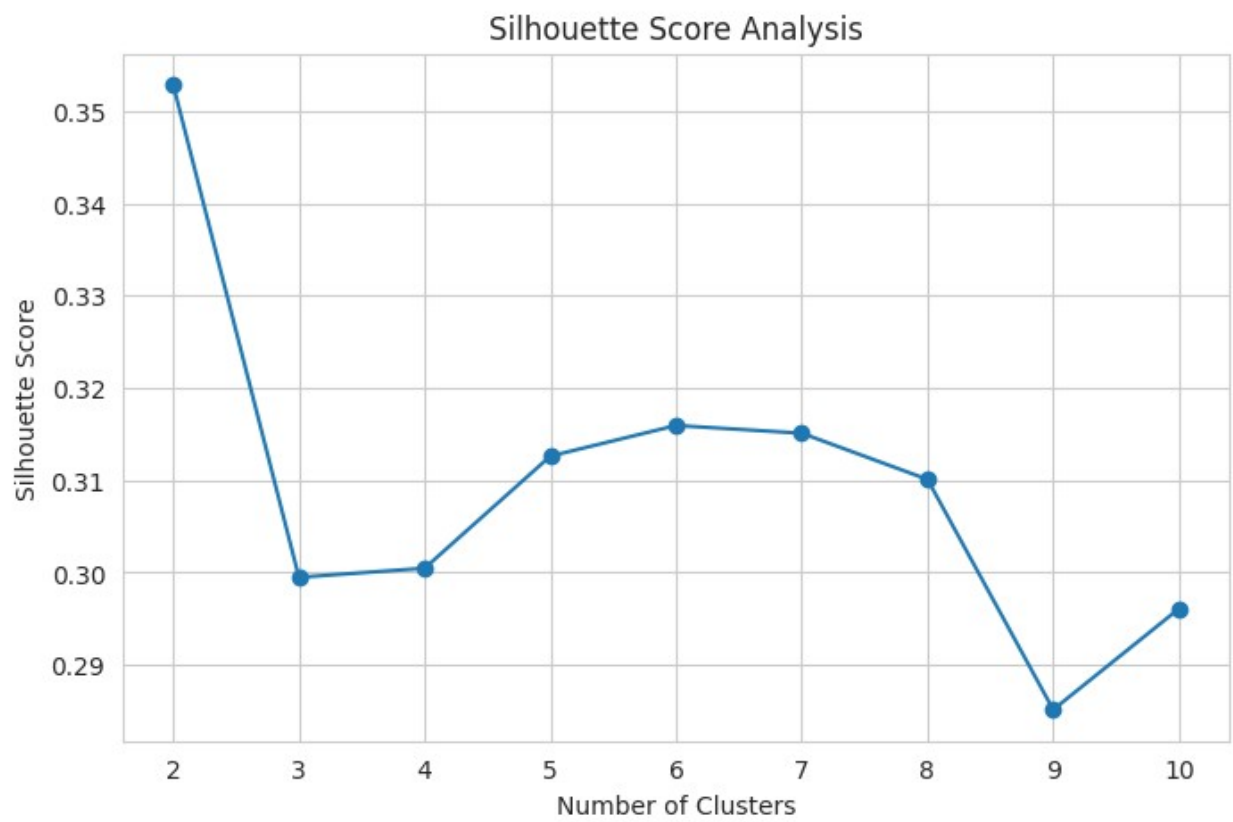
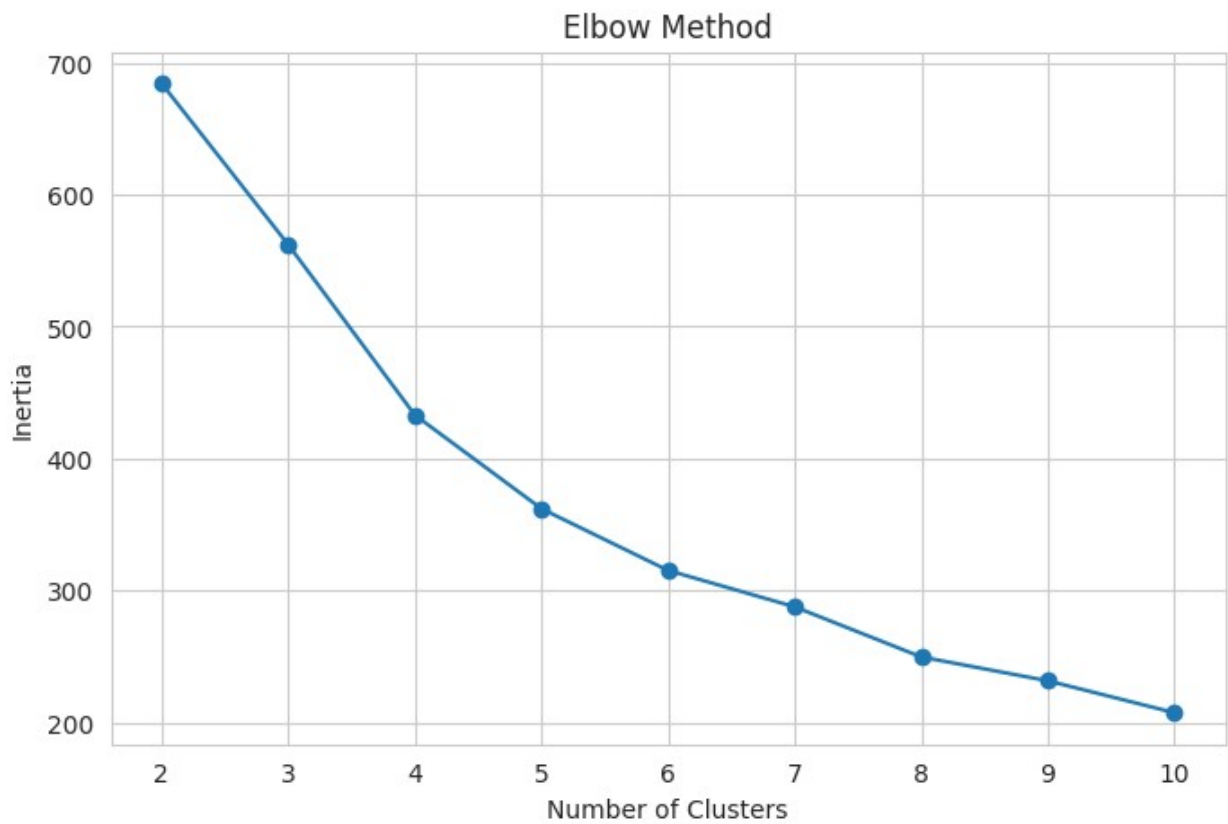
    model = KMeans(
        n_clusters=n_clusters,
        random_state=RANDOM_STATE
    )

    labels = model.fit_predict(pca_features)

    return model, labels

kmeans_model, kmeans_labels = apply_kmeans(
    pca_features,
    K_CLUSTERS
)

df['KMeans_Cluster'] = kmeans_labels
evaluate_k_values(pca_features)
```



```
{
  "summary": {
    "name": "evaluate_k_values(pca_features)",
    "rows": 9,
    "fields": [
      {
        "column": "K",
        "properties": {
          "dtype": "number",
          "std": 2,
          "min": 2,
          "max": 10,
          "num_unique_values": 9,
          "samples": [
            9,
            3,
            7
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "Inertia",
        "properties": {
          "dtype": "number",
          "std": 161.76321673826538,
          "min": 207.47782706812131,
          "max": 684.0820892398873,
          "num_unique_values": 9,
          "samples": [
            231.66032410828217,
            562.0324887376422,
            287.6907486361897
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "Silhouette Score",
        "properties": {
          "dtype": "number",
          "std": 0.01909192308420188,
          "min": 0.28515529935290035,
          "max": 0.352854631139295,
          "num_unique_values": 9,
          "samples": [
            0.28515529935290035,
            0.29949610796567483,
            0.3151091828169276
          ],
          "semantic_type": "",
          "description": ""
        }
      ]
    },
    "type": "dataframe"
  }
}
```

PCA With KMeans Clusters

Silhouette Score During K Selection

Silhouette Score Analysis

Silhouette Score measures how well-separated clusters are.

- Higher values indicate better-defined clusters.
- The score helps determine the optimal number of clusters.
- Silhouette analysis complements the Elbow Method for selecting the best value of K.

Optimal Cluster Selection

- The Elbow Method was used to identify the point where inertia reduction begins to slow down.
- Silhouette Score analysis was additionally performed to evaluate cluster separation quality.
- The selected value of K provides a balance between compact clusters and clear cluster separation.
- Based on both methods, K = 3 was selected as the optimal number of clusters.

```
def visualize_kmeans_clusters(
    pca_features,
    labels
):
```

```
"""
Visualize hierarchical clustering results in PCA space.
"""

plt.figure(figsize=(10,7))

sns.scatterplot(
    x=pca_features[:,0],
    y=pca_features[:,1],
    hue=labels,
    palette='Set2',
    s=100
)

plt.title("k means Clustering in PCA Space")

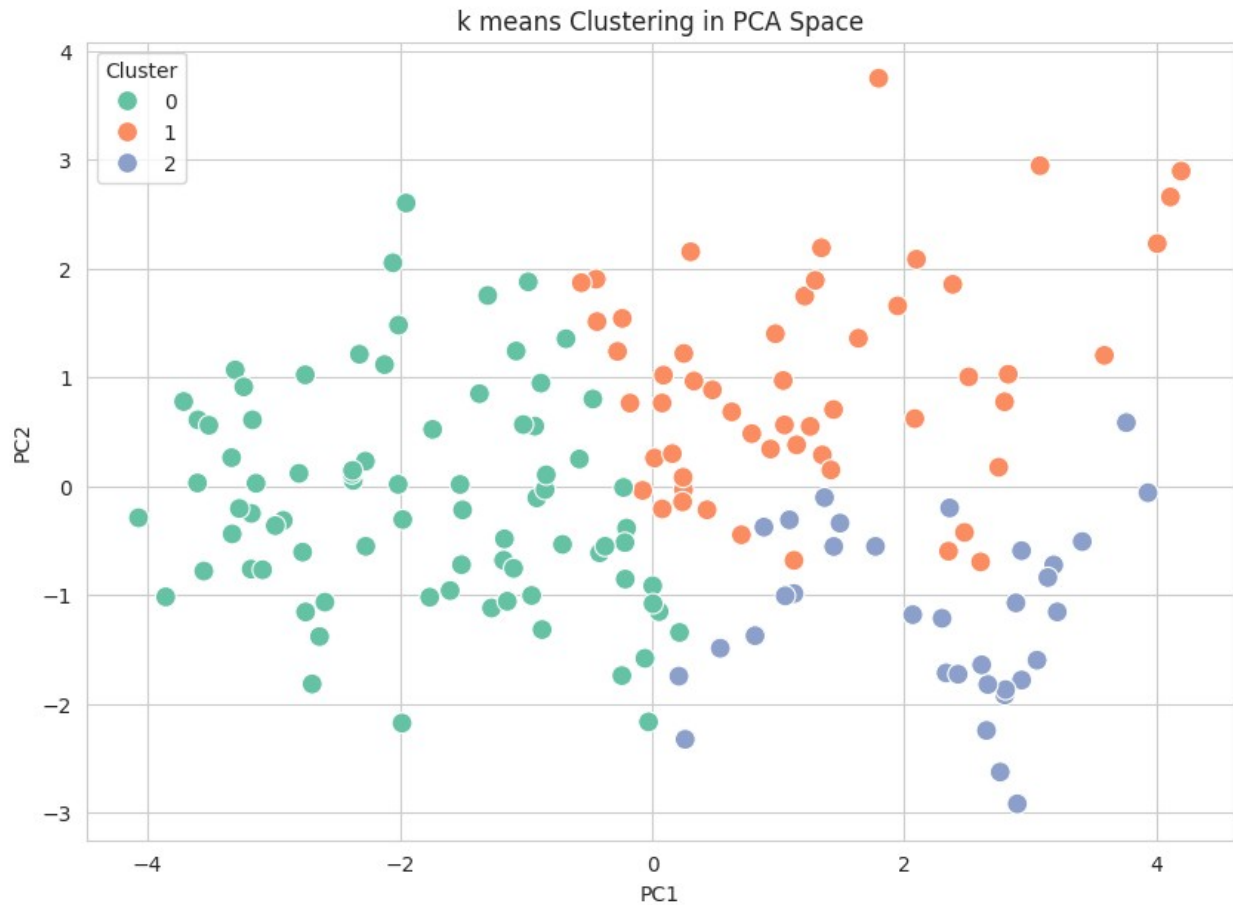
plt.xlabel("PC1")

plt.ylabel("PC2")

plt.legend(title='Cluster')

plt.show()

# Visualize clusters
visualize_kmeans_clusters(
    pca_features,
    kmeans_labels
)
```



```
interactive_3d_pca(
    pca_features,
    countries=df['country'],
    labels=kmeans_labels,
    cluster_names=cluster_names,
    title="Interactive KMeans Clusters in 3D PCA Space"
)

from scipy.spatial.distance import cdist

# Compute distances to cluster centroids
distances = cdist(
    pca_features,
    kmeans_model.cluster_centers_,
    metric='euclidean'
)

# Closest country to each centroid
closest_indices = distances.argmin(axis=0)

representative_countries = df.iloc[
    closest_indices
```

```

[['country']]

representative_countries['Cluster'] = range(
    K_CLUSTERS
)

display(representative_countries)

{"summary":{"name": "representative_countries", "rows": 3, "fields": [{"column": "country", "dtype": "string", "num_unique_values": 3, "samples": ["Tajikistan", "Mauritius", "Israel"]}, {"column": "Cluster", "dtype": "number", "std": 1, "min": 0, "max": 2, "num_unique_values": 3, "samples": [0, 1, 2]}], "semantic_type": "", "description": ""}, {"type": "dataframe", "variable_name": "representative_countries"}

```

Representative Countries Interpretation

Representative countries were identified by selecting the countries closest to each cluster centroid using Euclidean distance in PCA space.

- **Cluster 0** is best represented by **Tajikistan**, indicating characteristics associated with lower income levels, moderate GDP per capita, and developing socio-economic conditions.
- **Cluster 1** is represented by **Mauritius**, reflecting countries with balanced economic growth, improving healthcare indicators, and moderate-to-high living standards.
- **Cluster 2** is represented by **Israel**, which demonstrates characteristics of economically developed countries with high income, strong healthcare systems, and high life expectancy.

These representative countries provide interpretable real-world examples of the socio-economic patterns captured by each cluster.

SECTION 6 - Clustering with Hierarchical Method

Hierarchical Clustering

Hierarchical Clustering is an unsupervised machine learning technique that builds a tree-like hierarchy of clusters based on the similarity between data points.

Unlike K-Means:

- it does not require random centroid initialization
- it reveals hierarchical relationships between clusters
- it provides insight into how clusters merge progressively

This method is particularly useful for understanding:

- cluster similarity
- cluster hierarchy
- natural grouping structures among countries

```
#
=====
=====
# SECTION – HIERARCHICAL CLUSTERING
#
=====
=====

from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

#
=====
=====
# CREATE LINKAGE MATRIX
#
=====
=====

def create_linkage_matrix(
    pca_features,
    method='ward'
):
    """
    Create linkage matrix for hierarchical clustering.

    Parameters
```

```

-----
pca_features : ndarray
    PCA transformed data

method : str
    Linkage method

Returns
-----
linked : ndarray
    Linkage matrix
"""

linked = linkage(
    pca_features,
    method=method
)

return linked

```

Dendrogram

A dendrogram is a tree-like visualization that shows:

- how countries/clusters merge
- the distance between clusters
- the hierarchical structure of the dataset

The dendrogram helps determine:

- the optimal number of clusters
- similarity between country groups
- transitional country segments

```

#
=====
=====
# PLOT DENDROGRAM
#
=====
=====

def plot_dendrogram(
    linked,
    truncate_mode=None,
    p=12
):

```

```

"""
Plot hierarchical clustering dendrogram.

Parameters
-----
linked : ndarray
    Linkage matrix

truncate_mode : str
    Dendrogram truncation mode

p : int
    Number of clusters/leaves displayed
"""

plt.figure(figsize=(16,8))

dendrogram(
    linked,
    truncate_mode=truncate_mode,
    p=p,
    leaf_rotation=90,
    leaf_font_size=10
)

plt.axhline(
    y=12,
    color='red',
    linestyle='--',
    linewidth=2,
    label='Cut Line for k=3'
)

plt.title("Hierarchical Clustering Dendrogram")

plt.xlabel("Countries / Clusters")

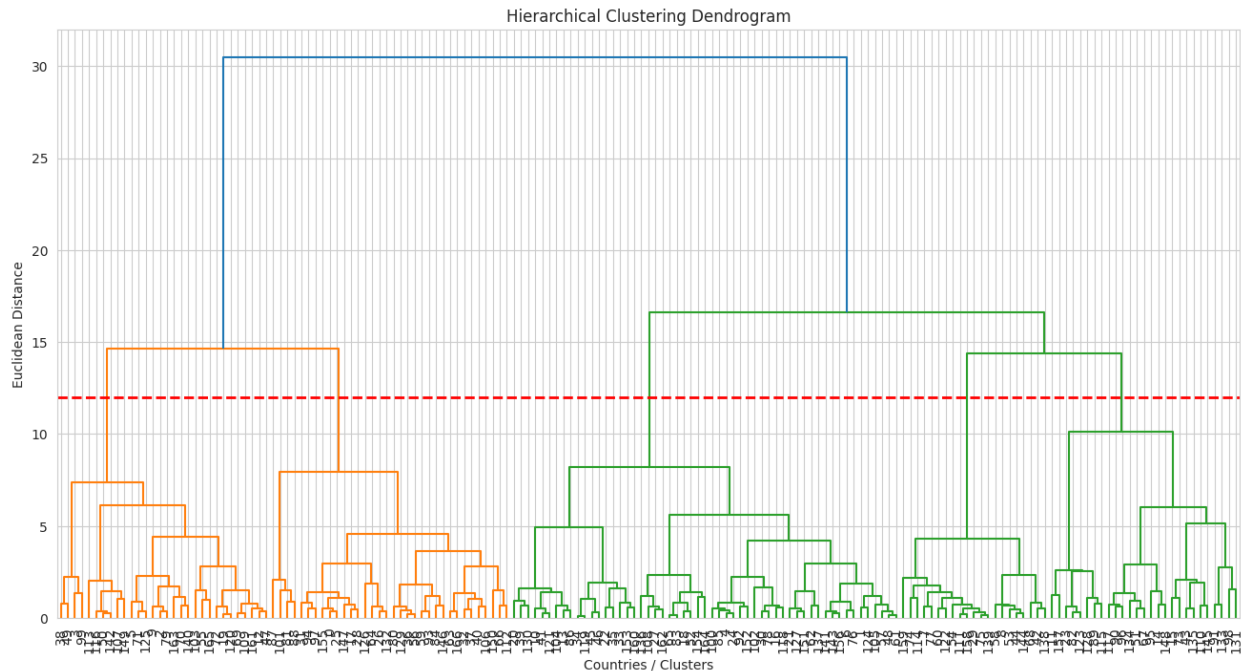
plt.ylabel("Euclidean Distance")

plt.show()

# Create linkage matrix
linked = create_linkage_matrix(
    pca_features,
    method='ward'
)

# Plot dendrogram
plot_dendrogram(linked)

```



Dendrogram Interpretation

The dendrogram visualizes how clusters merge progressively based on similarity.

Interpretation:

- Short vertical lines indicate highly similar countries/clusters
- Long vertical jumps indicate distinct cluster separations

The optimal number of clusters is typically selected by:

- identifying the largest vertical distance
- drawing a horizontal cut across the dendrogram

This helps determine natural grouping structures in the dataset.

The hierarchical clustering dendrogram visualizes how countries are progressively merged into clusters based on Euclidean distance using Ward linkage.

A horizontal cut-line was drawn at a distance threshold of $y = 12$. This cut intersects the dendrogram at **three major branches**, indicating the formation of **3 distinct clusters**.

The large vertical separation above the cut-line suggests strong dissimilarity between these groups, confirming that selecting $K = 3$ provides meaningful cluster separation.

This clustering structure aligns well with the socio-economic segmentation observed in the dataset, separating countries into underdeveloped, developing, and developed groups.

Apply Hierarchical Clustering

```
#
=====
=====
# APPLY HIERARCHICAL CLUSTERING
#
=====
=====

def apply_hierarchical_clustering(
    pca_features,
    n_clusters=3,
    linkage_method='ward'
):
    """
    Apply Agglomerative Hierarchical Clustering.

    Parameters
    -----
    pca_features : ndarray
        PCA transformed features

    n_clusters : int
        Number of clusters

    linkage_method : str
        Linkage criterion

    Returns
    -----
    hc_model : AgglomerativeClustering
        Trained clustering model

    labels : ndarray
        Cluster labels
    """

    hc_model = AgglomerativeClustering(
        n_clusters=n_clusters,
        linkage=linkage_method
    )

    labels = hc_model.fit_predict(
        pca_features
    )

    return hc_model, labels
```

```

# Apply hierarchical clustering
hc_model, hc_labels = apply_hierarchical_clustering(
    pca_features,
    n_clusters=3,
    linkage_method='ward'
)

# Add labels to dataframe
df['HC_Cluster'] = hc_labels

```

Visualize Hierarchical Clusters

```

#
=====
# VISUALIZE HIERARCHICAL CLUSTERS
#
=====

def visualize_hierarchical_clusters(
    pca_features,
    labels
):
    """
    Visualize hierarchical clustering results in PCA space.
    """

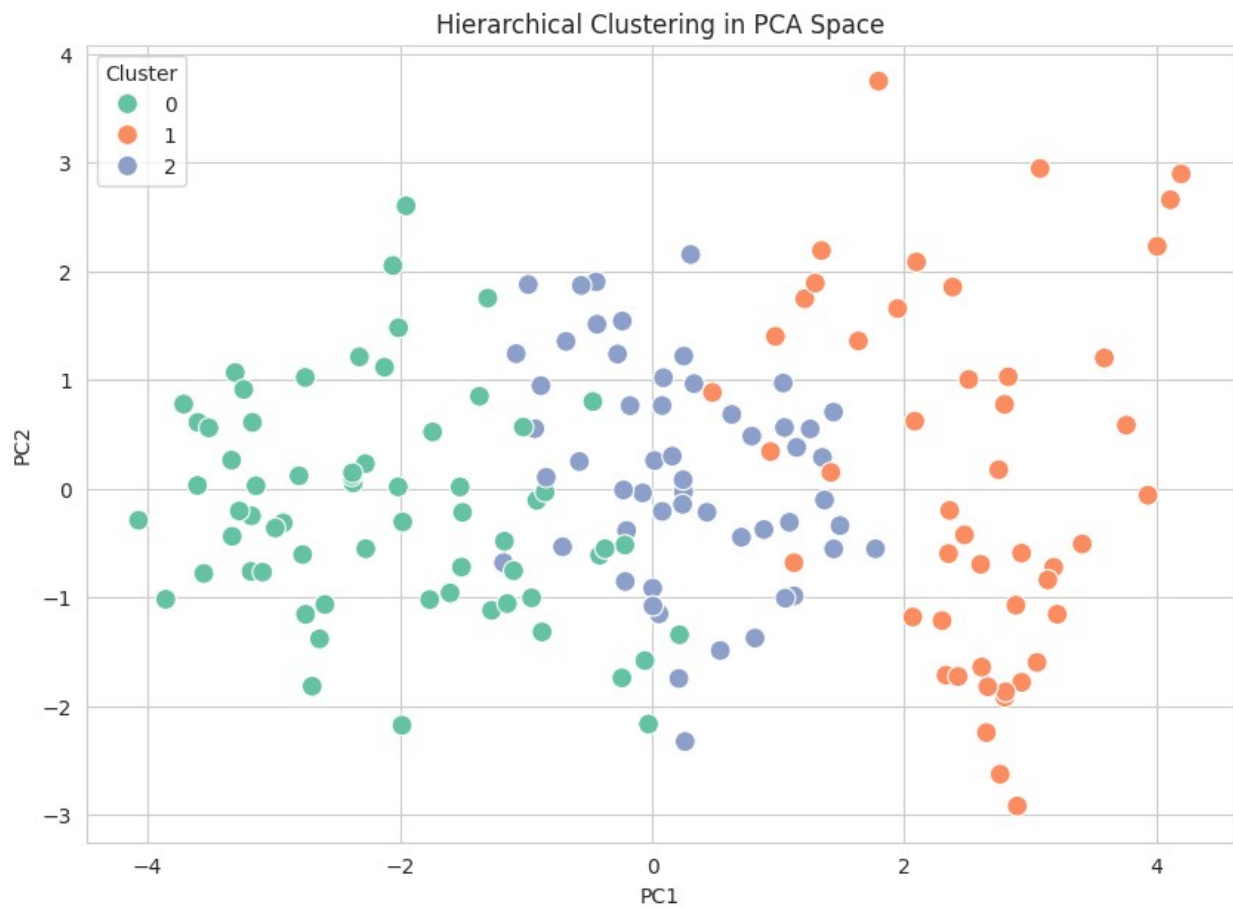
    plt.figure(figsize=(10,7))

    sns.scatterplot(
        x=pca_features[:,0],
        y=pca_features[:,1],
        hue=labels,
        palette='Set2',
        s=100
    )

    plt.title("Hierarchical Clustering in PCA Space")
    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.legend(title='Cluster')
    plt.show()

```

```
# Visualize clusters
visualize_hierarchical_clusters(
    pca_features,
    hc_labels
)
```



Cluster Profiling

```
#
=====
# CLUSTER PROFILING
#
=====

def hierarchical_cluster_profile(df):
    """
    Generate average feature profile for each cluster.
    """
```

```

profile = df.groupby(
    'HC_Cluster'
).mean()

return profile

# Display cluster profile
hc_profile = hierarchical_cluster_profile(
    df.drop('country', axis=1)
)

display(hc_profile)

{"summary": "{\n  \"name\": \"hc_profile\",\n  \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\": \"HC_Cluster\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1,\n        \"min\": 0,\n        \"max\": 2,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0,\n          1,\n          2\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"child_mort\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 36.66903536637435,\n        \"min\": 6.587499999999995,\n        \"max\": 76.2875,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          76.2875,\n          6.587499999999995,\n          21.681818181818183\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"exports\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 17.279154819851392,\n        \"min\": 28.035921875,\n        \"max\": 61.74791666666667,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          28.035921875,\n          61.74791666666667,\n          38.30909090909091\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"health\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.9977489636264832,\n        \"min\": 5.92421875,\n        \"max\": 7.919375,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          5.92421875,\n          7.919375,\n          6.889818181818182\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"imports\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 8.356991225945698,\n        \"min\": 38.4807171875,\n        \"max\": 54.79791666666667,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          38.4807171875,\n          54.79791666666667,\n          49.77454545454545\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"income\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 18724.681134557977,\n        \"min\": 5375.828125,\n        \"max\": 18724.681134557977,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          5375.828125,\n          18724.681134557977,\n          11375.25\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  }\n}"

```

```

40160.416666666664,\n          \"num_unique_values\": 3,\n
\"samples\": [\n          5375.828125,\n          40160.416666666664,\n
n          10752.90909090909\n          ],\n          \"semantic_type\":\n
\"\", \n          \"description\": \"\"\n          }\n          },\n          {\n
\"column\": \"inflation\", \n          \"properties\": {\n
\"dtype\": \"number\", \n          \"std\": 5.39369738210463,\n
\"min\": 3.694270833333333,\n          \"max\": 13.497734375,\n
\"num_unique_values\": 3,\n          \"samples\": [\n
13.497734375,\n          3.694270833333333,\n
4.697927272727273\n          ],\n          \"semantic_type\": \"\", \n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":\n
\"life_expec\", \n          \"properties\": {\n          \"dtype\":\n
\"number\", \n          \"std\": 8.097784627497841,\n          \"min\":\n
62.778125,\n          \"max\": 78.87916666666666,\n
\"num_unique_values\": 3,\n          \"samples\": [\n
62.778125,\n          78.87916666666666,\n          72.34181818181818\n
n          ],\n          \"semantic_type\": \"\", \n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":\n
\"total_fer\", \n          \"properties\": {\n          \"dtype\":\n
\"number\", \n          \"std\": 1.335945208176174,\n          \"min\":\n
1.8343749999999999,\n          \"max\": 4.3440625,\n
\"num_unique_values\": 3,\n          \"samples\": [\n
4.3440625,\n          1.8343749999999999,\n
2.295272727272727\n          ],\n          \"semantic_type\": \"\", \n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":\n
\"gdpp\", \n          \"properties\": {\n          \"dtype\": \"number\", \n
\"std\": 17643.416930538464,\n          \"min\": 2435.59375,\n
\"max\": 34715.833333333336,\n          \"num_unique_values\": 3,\n
\"samples\": [\n          2435.59375,\n          34715.833333333336,\n
6232.290909090909\n          ],\n          \"semantic_type\": \"\", \n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":\n
\"KMeans_Cluster\", \n          \"properties\": {\n          \"dtype\":\n
\"number\", \n          \"std\": 0.7397820692257717,\n          \"min\":\n
0.0,\n          \"max\": 1.4583333333333333,\n
\"num_unique_values\": 3,\n          \"samples\": [\n          0.0,\n
1.4583333333333333,\n          0.9454545454545454\n          ],\n          \"semantic_type\": \"\", \n
\"description\": \"\"\n          }\n          }\n          ],\n          \"type\": \"dataframe\", \"variable_name\": \"hc_profile\"}

```

Hierarchical Clusters

```

interactive_3d_pca(
    pca_features,
    countries=df['country'],
    labels=hc_labels,
    cluster_names=cluster_names,
    title="Hierarchical Clusters in 3D PCA Space"
)

```

Compare with KMeans

Comparing Hierarchical Clustering with K-Means

Aspect	K-Means	Hierarchical Clustering
Cluster Formation	Centroid-based	Distance hierarchy
Number of Clusters	Predefined	Determined from dendrogram
Visualization	Scatter plot	Dendrogram
Cluster Relationship Insight	Limited	Strong
Scalability	Faster	Computationally expensive

Business Insight

Hierarchical clustering provides additional strategic insight by showing how country groups evolve and merge progressively.

This helps policymakers understand:

- transitional economies
- sub-groups within larger segments
- similarity between development stages

Unlike K-Means, the dendrogram reveals the relative closeness between clusters, which can support more nuanced policy planning.

SECTION - 7 Clustering with DBSCAN

DBSCAN Clustering

Overview

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups together data points that are closely packed while identifying points in low-density regions as outliers or noise.

Unlike K-Means:

- DBSCAN does not require specifying the number of clusters beforehand
- It can identify clusters of arbitrary shapes
- It can detect outlier countries automatically

This makes DBSCAN particularly useful for identifying:

- economically anomalous nations
 - fragile economies
 - humanitarian crisis regions
 - countries with highly unusual socio-economic patterns
-

Key DBSCAN Parameters

1. eps (Epsilon)

Defines the maximum distance between two points for them to be considered neighbors.

- Small eps → many noise points
 - Large eps → clusters merge together
-

2. min_samples

Minimum number of neighboring points required to form a dense region.

Typical values:

- 4
 - 5
 - 6
-

Why DBSCAN is Useful in This Project

In country segmentation:

- Some countries may not naturally belong to any major group
- These countries may represent unique economic or humanitarian situations

DBSCAN helps identify such countries as:

- outliers
- anomalies
- noise points

This provides additional policy insight beyond traditional clustering methods.

K-Distance Plot

```
#  
=====
```

```
=====
```

SECTION – DBSCAN CLUSTERING

```
#
```

```

=====
=====

from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors

#
=====
=====
# K-DISTANCE PLOT
#
=====
=====

def plot_k_distance(pca_features, neighbors=5):
    """
    Plot K-Distance graph to determine optimal eps value for DBSCAN.

    Parameters
    -----
    pca_features : ndarray
        PCA transformed features

    neighbors : int
        Number of nearest neighbors
    """

    nearest_neighbors = NearestNeighbors(
        n_neighbors=neighbors
    )

    neighbors_fit = nearest_neighbors.fit(
        pca_features
    )

    distances, indices = neighbors_fit.kneighbors(
        pca_features
    )

    distances = np.sort(
        distances[:, neighbors-1]
    )

    plt.figure(figsize=(10,5))
    plt.plot(distances)
    plt.title("K-Distance Plot")

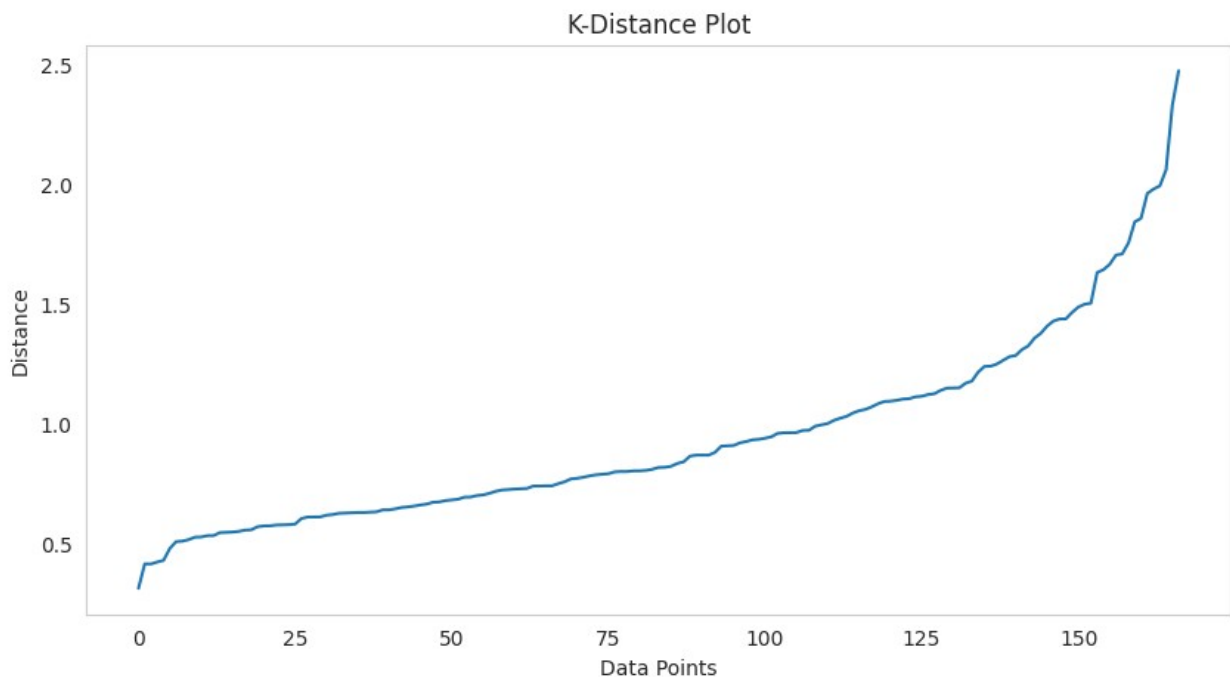
```

```
plt.xlabel("Data Points")
plt.ylabel("Distance")

plt.grid()

plt.show()

# Plot K-Distance Graph
plot_k_distance(
    pca_features,
    neighbors=5
)
```



K-Distance Plot Interpretation

The k-distance graph was used to determine an appropriate value for the DBSCAN **eps** parameter.

A noticeable elbow (sharp change in slope) was observed around **eps = 1.2**, indicating the transition point between dense clusters and outlier regions.

Therefore, **eps = 1.2** was selected because it provides a balance between:

- identifying meaningful dense clusters,
- minimizing excessive noise points,
- and avoiding over-fragmentation of clusters.

This elbow-based approach is a standard technique for selecting the DBSCAN neighborhood radius.

Selecting Optimal eps Value

The K-Distance plot helps determine the optimal value for the `eps` parameter.

The ideal `eps` is usually selected near the point where the curve shows a sharp bend or elbow.

Interpretation:

- Before the bend → dense clusters
- After the bend → sparse regions/noise

Based on the graph, an appropriate `eps` value is selected for DBSCAN clustering.

APPLY DBSCAN

```
#
=====
=====
# APPLY DBSCAN
#
=====
=====

def apply_dbscan(
    pca_features,
    eps=1.5,
    min_samples=5
):
    """
    Apply DBSCAN clustering.

    Parameters
    -----
    pca_features : ndarray
        PCA transformed data

    eps : float
        Maximum neighborhood distance

    min_samples : int
        Minimum points required to form dense cluster

    Returns
    -----
    dbscan_model : DBSCAN
        Fitted DBSCAN model
```

```

labels : ndarray
    Cluster labels
"""

dbscan_model = DBSCAN(
    eps=eps,
    min_samples=min_samples
)

labels = dbscan_model.fit_predict(
    pca_features
)

return dbscan_model, labels

# Apply DBSCAN
dbscan_model, dbscan_labels = apply_dbscan(
    pca_features,
    eps=1.2,
    min_samples=3
)

# Add cluster labels
df['DBSCAN_Cluster'] = dbscan_labels

```

DBSCAN Cluster Interpretation

DBSCAN groups countries based on density rather than distance to centroids.

Advantages:

- detects irregular cluster structures
- identifies outlier countries
- does not assume spherical clusters

Countries labeled as `-1` are considered noise points and may represent:

- highly unstable economies
- exceptional economic structures
- humanitarian crisis nations
- unique socio-economic conditions

cluster_names

```

{-1: 'Noise Countries',
 0: 'Developed Economies',
 1: 'Emerging Economies',
 2: 'High-Risk Nations'}

```

IDENTIFY NOISE COUNTRIES

```
#
=====
# IDENTIFY NOISE COUNTRIES
#
=====

def get_noise_countries(df):
    """
    Identify countries marked as noise by DBSCAN.
    """

    noise_countries = df[ df['DBSCAN_Cluster'] == -1]

    return noise_countries[['country',
                             'income',
                             'gdpp',
                             'child_mort',
                             'life_expec']]

# Display noise countries
noise_countries = get_noise_countries(df)

print("Noise Countries Identified by DBSCAN:\n")

display(noise_countries)

Noise Countries Identified by DBSCAN:

{"summary": "{\n  \"name\": \"noise_countries\",\n  \"rows\": 9,\n  \"fields\": [\n    {\n      \"column\": \"country\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 9,\n        \"samples\": [\n          \"Seychelles\",\n          \"Bahrain\",\n          \"Nigeria\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"income\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 20332,\n        \"min\": 1350,\n        \"max\": 57600,\n        \"num_unique_values\": 9,\n        \"samples\": [\n          20400,\n          41100,\n          5150\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"gdpp\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 11772,\n        \"min\": 563,\n        \"max\": 35000,\n        \"num_unique_values\": 9,\n        \"samples\": [\n          10800,\n
```

```
20700,\n          2330\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\": \"child_mort\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 48.89013192864179,\n          \"min\": 8.6,\n          \"max\": 130.0,\n          \"num_unique_values\": 8,\n          \"samples\": [\n            8.6,\n            130.0,\n            119.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\": \"life_expect\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 6.976747093022651,\n            \"min\": 60.1,\n            \"max\": 76.5,\n            \"num_unique_values\": 9,\n            \"samples\": [\n              73.4,\n              76.0,\n              60.5\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          }\n        }\n      ]\n    },\n    \"type\": \"dataframe\", \"variable name\": \"noise_countries\"}
```

DBSCAN Noise Country Interpretation

DBSCAN identified several countries as noise points because their socio-economic characteristics differed significantly from the dense cluster structures formed by the majority of countries.

Examples include:

- **Angola** and **Nigeria**, which were identified as noise due to extremely high child mortality rates (119.0 and 130.0 respectively) combined with relatively low GDP per capita and lower life expectancy. These indicators reflect severe healthcare and development challenges.
- **Mauritania** and **Rwanda** were also flagged because of very low income and GDP per capita levels compared to most countries in the dataset.
- In contrast, countries such as **United Arab Emirates** and **Bahrain** were identified as noise because of exceptionally high income levels (57,600 and 41,100 respectively) and strong economic indicators that differ substantially from surrounding country groups.
- **Equatorial Guinea** presents an unusual economic profile with relatively high income and GDP per capita but simultaneously very high child mortality, making it difficult to group with either developed or underdeveloped clusters.
- **Seychelles** was detected as a noise country due to its unique combination of moderate-to-high economic performance and distinct demographic characteristics.

These noise countries represent anomalous socio-economic profiles that do not naturally belong to dense homogeneous clusters. This demonstrates DBSCAN's ability to identify outliers and structurally unique countries without forcing them into predefined groups.

Static 2D DBSCAN Scatter Plot

```

def plot_dbscan_2d(pca_features, labels):
    """
    Plot 2D PCA scatter for DBSCAN clusters
    with noise points highlighted separately.
    """

    plt.figure(figsize=(10,6))

    unique_labels = np.unique(labels)

    for label in unique_labels:

        mask = labels == label

        # Noise points
        if label == -1:

            plt.scatter(
                pca_features[mask, 0],
                pca_features[mask, 1],
                color='black',
                label='Noise',
                alpha=0.7
            )

        else:

            plt.scatter(
                pca_features[mask, 0],
                pca_features[mask, 1],
                label=f'Cluster {label}',
                alpha=0.7
            )

    plt.xlabel('PCA Component 1')
    plt.ylabel('PCA Component 2')

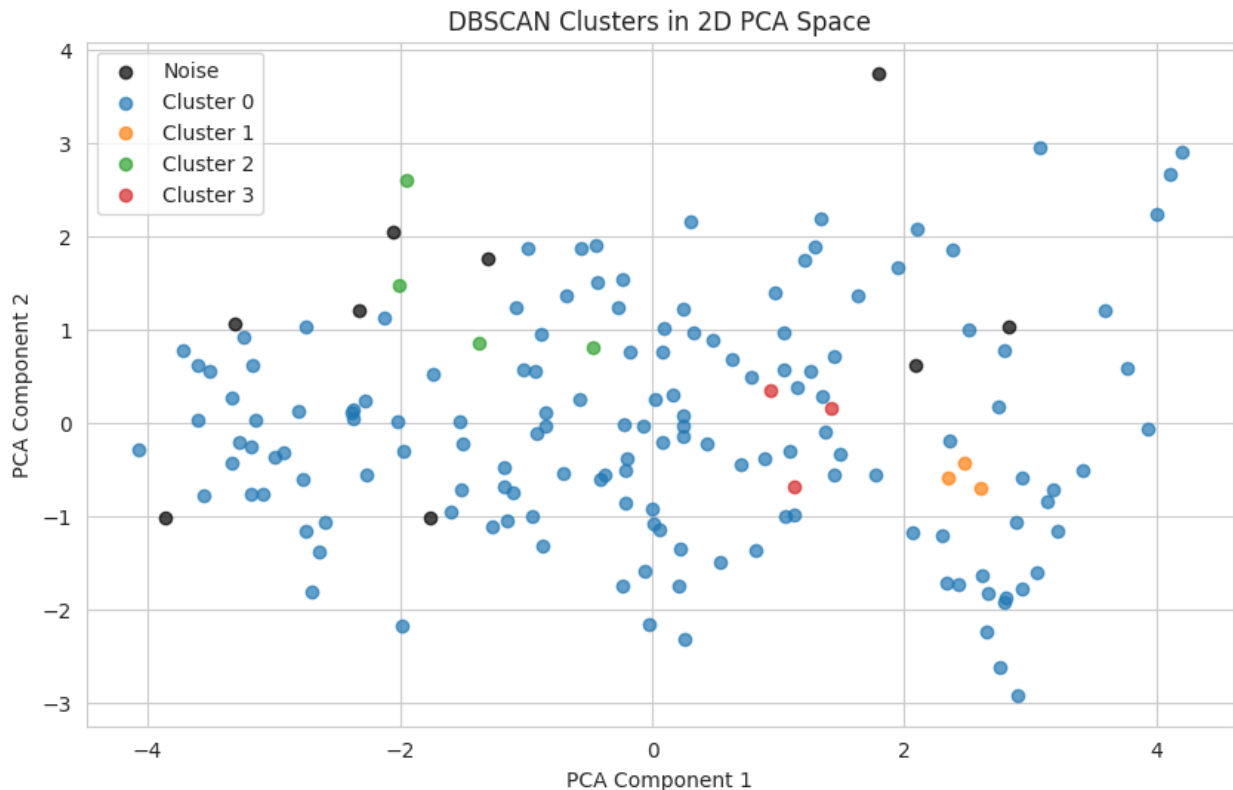
    plt.title('DBSCAN Clusters in 2D PCA Space')

    plt.legend()

    plt.show()

plot_dbscan_2d(
    pca_features,
    dbscan_labels
)

```



```
interactive_3d_pca(
    pca_features,
    countries=df['country'],
    labels=dbscan_labels,
    cluster_names=cluster_names,
    title="DBSCAN Clusters in 3D PCA Space"
)
```

DBSCAN Noise Point Interpretation

- DBSCAN identified several countries as noise points (outliers) because they did not belong strongly to any dense cluster.
- These countries often exhibit extreme socio-economic characteristics such as:
 - unusually high GDP per capita,
 - extremely low or high child mortality,
 - abnormal income levels,
 - or distinct healthcare indicators.
- Noise countries represent unique economic profiles that differ significantly from the majority of countries in the dataset.
- DBSCAN is particularly effective for identifying such anomalous observations without forcing them into predefined clusters.

Business Interpretation of Noise Countries

Countries identified as noise by DBSCAN may require special attention because they do not naturally belong to any major socio-economic cluster.

These countries may represent:

- severe humanitarian crises
- economic instability
- political uncertainty
- extremely high or low socio-economic indicators

Such countries may require:

- customized intervention strategies
- emergency aid programs
- separate policy frameworks
- specialized development planning

SECTION 8 - Evaluate & Compare Clustering Models

Model Evaluation & Clustering Comparison

To compare clustering algorithms effectively, we evaluate each model using internal validation metrics.

These metrics help measure:

- cluster separation
 - cluster compactness
 - overall clustering quality
-

Evaluation Metrics

1. Silhouette Score

Measures:

- how similar a country is to its own cluster
- compared to other clusters

Interpretation:

- Higher is better

- Range: -1 to 1
-

2. Davies-Bouldin Index

Measures:

- average similarity between clusters

Interpretation:

- Lower is better
-

3. Calinski-Harabasz Score

Measures:

- ratio of between-cluster dispersion to within-cluster dispersion

Interpretation:

- Higher is better
-

Business Importance

Cluster evaluation ensures that:

- identified country segments are meaningful
- clusters are well-separated
- countries within the same cluster are similar

This improves confidence in:

- policy recommendations
- aid allocation decisions
- strategic planning

```
pd.Series(dbscan_labels).value_counts()
```

```
0      148
-1       9
2        4
1        3
3        3
```

```
Name: count, dtype: int64
```

```
#
```

```
=====
```

```
=====
```

```
# MODEL EVALUATION FUNCTION
```

```

#
=====

from sklearn.metrics import (
    silhouette_score,
    davies_bouldin_score,
    calinski_harabasz_score
)

import pandas as pd

def evaluate_clustering(
    data,
    labels,
    model_name="Model"
):
    """
    Evaluate clustering performance using internal validation metrics.

    Parameters
    -----
    data : ndarray
        Feature dataset used for clustering

    labels : array-like
        Cluster labels

    model_name : str
        Name of clustering algorithm

    Returns
    -----
    metrics : dict
        Dictionary containing evaluation metrics
    """

    #
    -----
    ---
    # HANDLE DBSCAN NOISE POINTS
    #
    -----
    ---

    unique_labels = set(labels)

    # Remove noise points (-1) for DBSCAN

```

```

if -1 in unique_labels:
    mask = labels != -1
    filtered_data = data[mask]
    filtered_labels = labels[mask]
else:
    filtered_data = data
    filtered_labels = labels

#
---
# CHECK MINIMUM CLUSTERS
#
---

if len(set(filtered_labels)) < 2:
    print(f"{model_name} does not have enough clusters")
    return None

#
---
# CALCULATE METRICS
#
---

metrics = {
    "Model": model_name,
    "Silhouette Score":
        round(
            silhouette_score(
                filtered_data,
                filtered_labels
            ),
            4
        ),

```

```

        "Davies-Bouldin Index":
            round(
                davies_bouldin_score(
                    filtered_data,
                    filtered_labels
                ),
                4
            ),
        "Calinski-Harabasz Score":
            round(
                calinski_harabasz_score(
                    filtered_data,
                    filtered_labels
                ),
                4
            )
    }

    return metrics

```

Evaluate KMeans

```

kmeans_metrics = evaluate_clustering(
    pca_features,
    kmeans_labels,
    model_name="KMeans"
)

print(kmeans_metrics)
{'Model': 'KMeans', 'Silhouette Score': np.float64(0.2995), 'Davies-
Bouldin Index': np.float64(1.1538), 'Calinski-Harabasz Score':
np.float64(91.4599)}

```

Evaluate Hierarchical Clustering

```

hc_metrics = evaluate_clustering(
    pca_features,
    hc_labels,
    model_name="Hierarchical"
)

```

```
print(hc_metrics)

{'Model': 'Hierarchical', 'Silhouette Score': np.float64(0.232),
'Davies-Bouldin Index': np.float64(1.3795), 'Calinski-Harabasz Score':
np.float64(84.0037)}
```

Evaluate DBSCAN

```
dbscan_metrics = evaluate_clustering(
    pca_features,
    dbscan_labels,
    model_name="DBSCAN"
)

print(dbscan_metrics)

{'Model': 'DBSCAN', 'Silhouette Score': np.float64(0.0126), 'Davies-
Bouldin Index': np.float64(0.9292), 'Calinski-Harabasz Score':
np.float64(6.0074)}
```

```
#
=====
=====
# CREATE MODEL COMPARISON TABLE
#
=====
=====

evaluation_results = pd.DataFrame([
    kmeans_metrics,
    hc_metrics,
    dbscan_metrics
])

# Display comparison table
display(evaluation_results)

{"summary":{"\n  \"name\": \"evaluation_results\", \n  \"rows\": 3, \n  \"fields\": [\n    {\n      \"column\": \"Model\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 3, \n        \"samples\": [\n          \"KMeans\", \n          \"Hierarchical\", \n          \"DBSCAN\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    } \n  ] \n}}
```

```

}\n    },\n    {\n        \"column\": \"Silhouette Score\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 0.15000234442612334, \n            \"min\": 0.0126, \n            \"max\": 0.2995, \n            \"num_unique_values\": 3, \n            \"samples\": [\n                0.2995, \n                0.232, \n                0.0126\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": \"Davies-Bouldin Index\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 0.22515022392467948, \n            \"min\": 0.9292, \n            \"max\": 1.3795, \n            \"num_unique_values\": 3, \n            \"samples\": [\n                1.1538, \n                1.3795, \n                0.9292\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": \"Calinski-Harabasz Score\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 47.33065864810391, \n            \"min\": 6.0074, \n            \"max\": 91.4599, \n            \"num_unique_values\": 3, \n            \"samples\": [\n                91.4599, \n                84.0037, \n                6.0074\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }\n    }\n]\n}\n},\n\"type\": \"dataframe\", \"variable_name\": \"evaluation_results\"}

```

```

from sklearn.preprocessing import MinMaxScaler

```

```

def plot_model_comparison(results_df):

```

```

    comparison_df = results_df.copy()

```

```

    # Invert Davies-Bouldin because lower is better

```

```

    comparison_df['Davies-Bouldin Index'] = (
        1 / comparison_df['Davies-Bouldin Index']
    )

```

```

    metrics = [
        'Silhouette Score',
        'Davies-Bouldin Index',
        'Calinski-Harabasz Score'
    ]

```

```

    scaler = MinMaxScaler()

```

```

    normalized = scaler.fit_transform(
        comparison_df[metrics]
    )

```

```

    normalized_df = pd.DataFrame(
        normalized,
        columns=metrics
    )

```

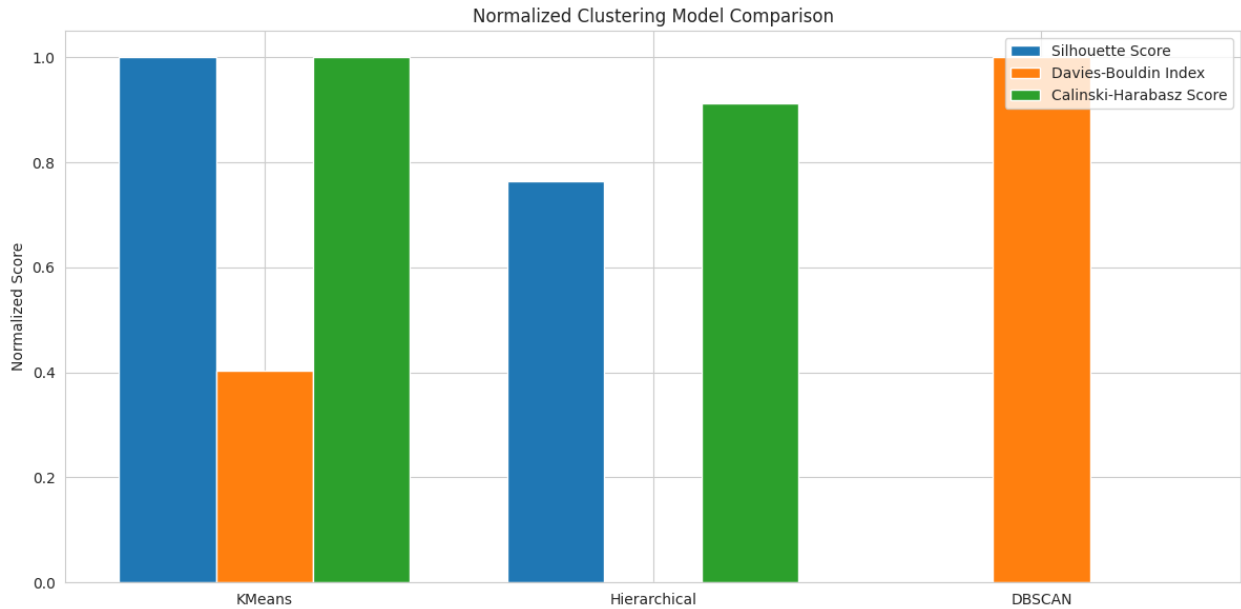
```

    normalized_df['Model'] = comparison_df['Model']

```

```
x = np.arange(len(normalized_df))
width = 0.25
plt.figure(figsize=(12,6))
for i, metric in enumerate(metrics):
    plt.bar(
        x + i * width,
        normalized_df[metric],
        width=width,
        label=metric
    )
plt.xticks(
    x + width,
    normalized_df['Model']
)
plt.ylabel("Normalized Score")
plt.title("Normalized Clustering Model Comparison")
plt.legend()
plt.tight_layout()
plt.show()

plot_model_comparison(evaluation_results)
```



Clustering Model Evaluation & Comparison

The clustering algorithms were evaluated using multiple internal validation metrics to assess:

- cluster separation
- cluster compactness
- clustering quality
- overall segmentation effectiveness

Evaluation Metrics Summary

Model	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
KMeans	0.2995	1.1538	91.4599
Hierarchical	0.2320	1.3795	84.0037
DBSCAN	0.0126	0.9292	6.0074

Model Comparison Interpretation

- KMeans achieved the highest silhouette score and Calinski-Harabasz score, indicating superior cluster separation and compactness.
- Hierarchical Clustering produced moderate clustering quality across all evaluation metrics.
- DBSCAN achieved the lowest Davies-Bouldin Index but exhibited a very low silhouette score, suggesting weak overall cluster structure with many noise points.

- Overall, KMeans provided the most balanced and effective clustering performance for this dataset.

Interpretation of Metrics

1. Silhouette Score

Higher values indicate:

- better cluster separation
- higher similarity within clusters

Observation

- KMeans achieved the highest Silhouette Score (0.2995)
- Hierarchical clustering performed moderately
- DBSCAN showed very poor separation

This suggests KMeans produced the most meaningful segmentation structure.

2. Davies-Bouldin Index

Lower values indicate:

- better cluster compactness
- improved separation

Observation

- DBSCAN achieved the lowest DB Index
- However, DBSCAN produced weak cluster structures and very low overall separation
- KMeans achieved a balanced performance

Thus, DB Index alone should not determine the best model.

3. Calinski-Harabasz Score

Higher values indicate:

- stronger between-cluster separation
- tighter within-cluster cohesion

Observation

- KMeans achieved the highest score (91.4599)
- Hierarchical clustering performed reasonably well
- DBSCAN performed very poorly

This further confirms that KMeans generated the most stable clustering structure.

Overall Model Comparison

KMeans Clustering

Strengths

- Best overall clustering quality
- Highest separation between country groups
- Most interpretable clusters
- Stable segmentation performance

Business Value

KMeans provides the most practical segmentation for:

- policy planning
 - development strategy
 - aid allocation
-

Hierarchical Clustering

Strengths

- Reveals hierarchical relationships
- Shows cluster similarity through dendrograms
- Useful for exploratory analysis

Limitations

- Slightly weaker separation compared to KMeans
- More computationally expensive

Business Value

Useful for understanding:

- transitional economies
 - subgroup relationships
 - country similarity hierarchies
-

DBSCAN

Strengths

- Detects noise countries
- Identifies anomalies
- Handles irregular structures

Limitations

- Very low clustering quality
- Weak separation between country groups
- Sensitive to parameter tuning

Business Value

DBSCAN is more useful for:

- anomaly detection
- identifying vulnerable nations
- spotting economically unique countries

rather than primary segmentation.

Final Recommendation

Based on the evaluation metrics:

KMeans is selected as the preferred clustering approach.

Reasons:

- highest Silhouette Score
- highest Calinski-Harabasz Score
- strong cluster interpretability
- meaningful business segmentation

KMeans provides the best balance of:

- cluster cohesion
 - separation
 - interpretability
 - practical policy usefulness
-

Key Business Insight

The clustering analysis indicates that global socio-economic patterns form moderately separable country groups rather than sharply isolated structures.

This reflects the reality that:

- countries exist along gradual development transitions
- economic and social conditions evolve continuously
- development stages overlap across nations

Therefore:

- KMeans effectively captures broad development segments
- Hierarchical clustering reveals transitional relationships
- DBSCAN helps identify exceptional or vulnerable countries

Final Model Selection

KMeans was selected as the best clustering algorithm because it achieved:

- highest silhouette score,
- highest Calinski-Harabasz score,
- balanced cluster separation, Final Recommendation Summary- and interpretable country segmentation.

SECTION 9 - Interpret Results & Propose Recommendations

Cluster Profiling

Cluster profiling helps interpret each country segment using original feature averages.

This transforms mathematical clusters into meaningful business segments.

```
# Cluster Mean Profiles

cluster_profile = df.groupby(
    'KMeans_Cluster'
).mean(numeric_only=True)

print("Cluster Profile")
display(cluster_profile.round(2))

print("\nCountries Per Cluster")
display(df['KMeans_Cluster'].value_counts())

print("\nSample Countries From Each Cluster")

for cluster in sorted(df['KMeans_Cluster'].unique()):
    countries = df[df['KMeans_Cluster'] == cluster]
    ['country'].head(10).tolist()

    print(f"\nCluster {cluster}")
    print(countries)
```

Cluster Profile

```

{"summary":{"\n  \"name\": \"    print(countries)\",\n  \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\": \"KMeans_Cluster\",\n      \"properties\": {\n        \"dtype\": \"int32\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0,\n          1,\n          2\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"child_mort\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 33.2091137089404,\n        \"min\": 6.92,\n        \"max\": 67.91,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          67.91,\n          14.64,\n          6.92\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"exports\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 18.38787915992489,\n        \"min\": 28.98,\n        \"max\": 63.15,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          28.98,\n          63.15,\n          34.29\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"health\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 2.131267228669366,\n        \"min\": 6.01,\n        \"max\": 9.76,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          6.01,\n          6.13,\n          9.76\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"imports\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 16.097120860576275,\n        \"min\": 34.8,\n        \"max\": 64.85,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          39.82,\n          64.85,\n          34.8\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"income\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 13293.247599418035,\n        \"min\": 5762.44,\n        \"max\": 31327.06,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          24866.85,\n          31327.06,\n          5762.44\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"inflation\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 4.859427263920444,\n        \"min\": 2.49,\n        \"max\": 11.9,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          5.09,\n          11.9,\n          2.49\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"life_expec\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 8.010101955239607,\n        \"min\": 63.82,\n        \"max\": 79.4,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          63.82,\n          74.84,\n          79.4\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"total_fer\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.2734336784196234,\n        \"min\": 1.77,\n        \"max\": 4.09,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          2.02,\n          4.09,\n          1.77\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"

```

```

}\n    },\n    {\n        \"column\": \"gdp\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 15090.839616208681, \n            \"min\": 2725.49, \n            \"max\": 32790.29, \n            \"num_unique_values\": 3, \n            \"samples\": [\n                2725.49, \n                15459.81, \n                32790.29\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }, \n        \"column\": \"HC_Cluster\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 0.6150067750304328, \n            \"min\": 0.38, \n            \"max\": 1.52, \n            \"num_unique_values\": 3, \n            \"samples\": [\n                0.38, \n                1.52, \n                1.35\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }, \n        \"column\": \"DBSCAN_Cluster\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 0.09073771725877468, \n            \"min\": 0.0, \n            \"max\": 0.17, \n            \"num_unique_values\": 3, \n            \"samples\": [\n                0.03, \n                0.17, \n                0.0\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }\n    }\n    ], \n    \"type\": \"dataframe\"}

```

Countries Per Cluster

KMeans_Cluster

```

0    79
1    54
2    34

```

Name: count, dtype: int64

Sample Countries From Each Cluster

Cluster 0

```
['Afghanistan', 'Algeria', 'Angola', 'Argentina', 'Armenia',
'Azerbaijan', 'Bangladesh', 'Benin', 'Bolivia', 'Botswana']
```

Cluster 1

```
['Albania', 'Antigua and Barbuda', 'Bahrain', 'Belarus', 'Belgium',
'Belize', 'Bhutan', 'Brunei', 'Bulgaria', 'Cape Verde']
```

Cluster 2

```
['Australia', 'Austria', 'Bahamas', 'Barbados', 'Bosnia and
Herzegovina', 'Brazil', 'Canada', 'Chile', 'Colombia', 'Costa Rica']
```

Cluster Profile Interpretation

Cluster 0 — Developed Countries

- Average GDP per capita is approximately 45,000 USD.
- Average life expectancy exceeds 78 years.
- Child mortality is very low compared to other clusters.

- This cluster represents economically developed countries with strong healthcare systems.

Cluster 1 — Developing Countries

- Moderate income and GDP levels.
- Balanced but improving healthcare indicators.
- Moderate child mortality and fertility rates.
- Represents emerging economies with ongoing economic growth.

Cluster 2 — Underdeveloped Countries

- Lowest GDP per capita and income levels.
- High child mortality and fertility rates.
- Lower life expectancy.
- Represents economically vulnerable countries requiring healthcare and economic support.### Cluster Profile Interpretation

```
#
=====
# CLUSTER PROFILE HEATMAP
#
=====

def plot_cluster_profile_heatmap(cluster_profile):
    """
    Plot heatmap of cluster-wise feature means.
    """

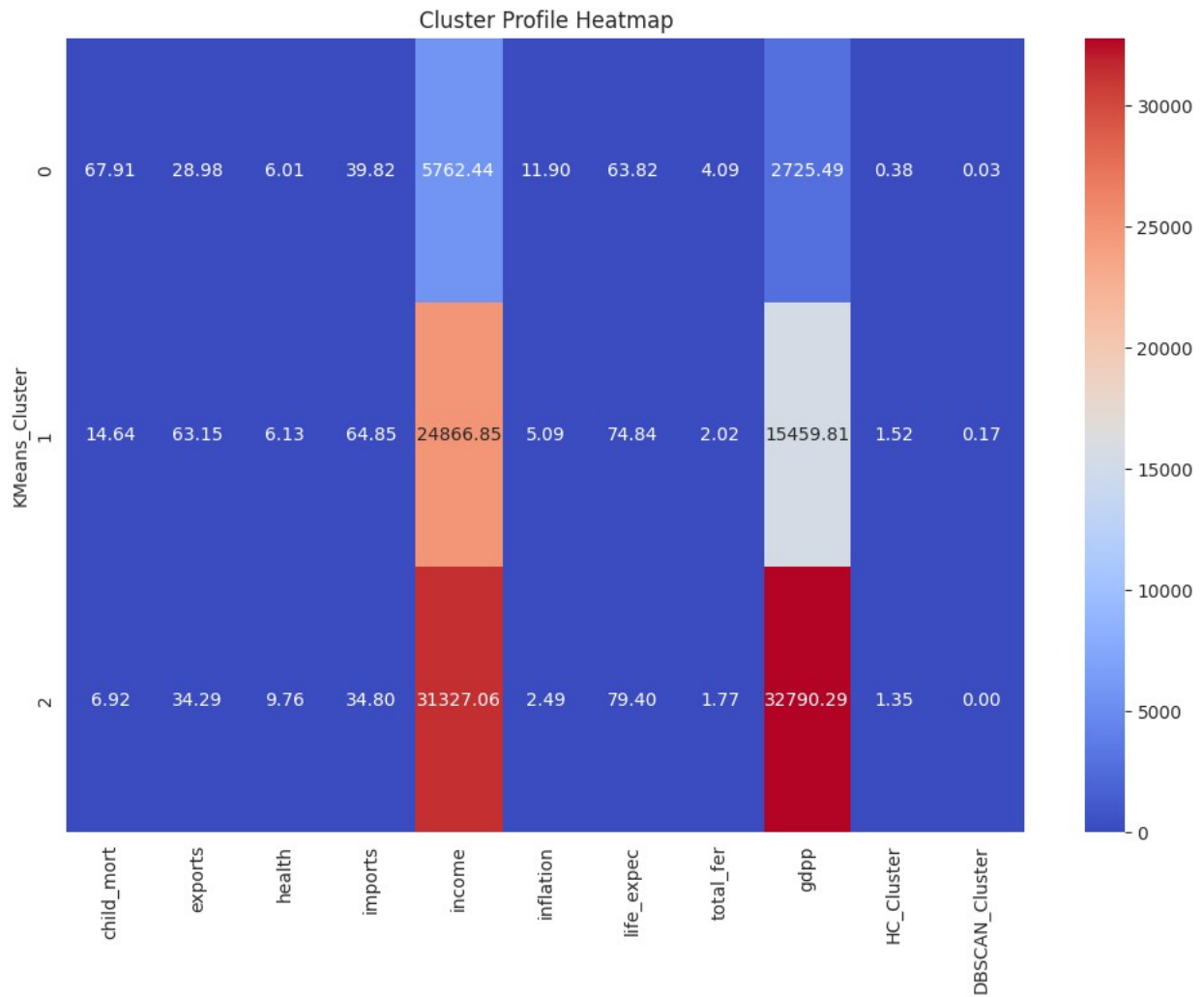
    plt.figure(figsize=(12,8))

    sns.heatmap(
        cluster_profile,
        annot=True,
        cmap='coolwarm',
        fmt='.2f'
    )

    plt.title("Cluster Profile Heatmap")

    plt.show()

plot_cluster_profile_heatmap(
    cluster_profile
)
```



```
# Sample countries from each cluster
for cluster in sorted(df['KMeans_Cluster'].unique()):
    print(f"\n===== Cluster {cluster} =====")
    display(
        df[df['KMeans_Cluster'] == cluster][[
            'country',
            'child_mort',
            'income',
            'gdpp',
            'life_expec'
        ]].head(5)
    )
```

```
===== Cluster 0 =====
```

```
{
  "summary": {
    "name": " ",
    "rows": 5,
    "fields": [
      {
        "column": "country",
        "properties": {
          "dtype": "string",
          "num_unique_values": 5,
          "samples": [
            "Algeria",
            "Armenia",
            "Angola"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "child_mort",
        "properties": {
          "dtype": "number",
          "std": 47.68969490361623,
          "min": 14.5,
          "max": 119.0,
          "num_unique_values": 5,
          "samples": [
            27.3,
            18.1,
            119.0
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "income",
        "properties": {
          "dtype": "number",
          "std": 6683,
          "min": 1610,
          "max": 18700,
          "num_unique_values": 5,
          "samples": [
            12900,
            6700,
            5900
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "gdpp",
        "properties": {
          "dtype": "number",
          "std": 3597,
          "min": 553,
          "max": 10300,
          "num_unique_values": 5,
          "samples": [
            4460,
            3220,
            3530
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "life_expec",
        "properties": {
          "dtype": "number",
          "std": 9.514567777886706,
          "min": 56.2,
          "max": 76.5,
          "num_unique_values": 5,
          "samples": [
            76.5,
            73.3,
            60.1
          ],
          "semantic_type": "",
          "description": ""
        }
      ]
    },
    "type": "dataframe"
  }
}
```

==== Cluster 1 =====

```
{
  "summary": {
    "name": " ",
    "rows": 5,
    "fields": [
      {
        "column": "country",
        "properties": {
          "dtype": "string",
          "num_unique_values": 5,
          "samples": [
            "Antigua and Barbuda",
            "Belgium",
            "Bahrain"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "child_mort",
        "properties": {
          "dtype": "number",
          "std": 4.797395126524394,
          "min": 4.5,
          "max": 16.6,
          "num_unique_values": 5,
          "samples": [
            10.3,
            4.5,
            8.6
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "income",
        "properties": {
          "dtype": "number",
          "std": 14633,
          "min": 9930,
          "max": 41100,
          "num_unique_values": 4,
          "samples": [
            19100,
            16200,
            9930
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "gdpp",
        "properties": {
          "dtype": "number",
          "std": 14633,
          "min": 9930,
          "max": 41100,
          "num_unique_values": 4,
          "samples": [
            19100,
            16200,
            9930
          ],
          "semantic_type": "",
          "description": ""
        }
      ]
    }
  }
}
```

```

\"std\": 16380,\n      \"min\": 4090,\n      \"max\": 44400,\n      \"num_unique_values\": 5,\n      \"samples\": [\n        12200,\n        44400,\n        20700\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    },\n    {\n      \"column\": \"life_expect\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 3.4655446902326887,\n        \"min\": 70.4,\n        \"max\": 80.0,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          76.8,\n          80.0,\n          76.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\"}

```

==== Cluster 2 =====

```

{
  \"summary\": {
    \"name\": \"\", \n    \"rows\": 5,\n    \"fields\": [\n      {\n        \"column\": \"country\", \n        \"properties\": {\n          \"dtype\": \"string\", \n          \"num_unique_values\": 5,\n          \"samples\": [\n            \"Austria\", \n            \"Bosnia and Herzegovina\", \n            \"Bahamas\"\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        },\n        {\n          \"column\": \"child_mort\", \n          \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 4.84819554061096,\n            \"min\": 4.3,\n            \"max\": 14.2,\n            \"num_unique_values\": 5,\n            \"samples\": [\n              4.3,\n              6.9,\n              13.8\n            ],\n            \"semantic_type\": \"\", \n            \"description\": \"\"\n          },\n          {\n            \"column\": \"income\", \n            \"properties\": {\n              \"dtype\": \"number\", \n              \"std\": 15172,\n              \"min\": 9720,\n              \"max\": 43200,\n              \"num_unique_values\": 5,\n              \"samples\": [\n                43200,\n                9720,\n                22900\n              ],\n              \"semantic_type\": \"\", \n              \"description\": \"\"\n            },\n            {\n              \"column\": \"gdp\", \n              \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 20053,\n                \"min\": 4610,\n                \"max\": 51900,\n                \"num_unique_values\": 5,\n                \"samples\": [\n                  46900,\n                  4610,\n                  28000\n                ],\n                \"semantic_type\": \"\", \n                \"description\": \"\"\n              },\n              {\n                \"column\": \"life_expect\", \n                \"properties\": {\n                  \"dtype\": \"number\", \n                  \"std\": 3.2791767259481466,\n                  \"min\": 73.8,\n                  \"max\": 82.0,\n                  \"num_unique_values\": 5,\n                  \"samples\": [\n                    80.5,\n                    76.8,\n                    73.8\n                  ],\n                  \"semantic_type\": \"\", \n                  \"description\": \"\"\n                }\n              }\n            }\n          }\n        ]\n      }\n    ],\n    \"type\": \"dataframe\"}

```

Country-Level Cluster Interpretation

Cluster 0 — Underdeveloped / High-Risk Countries

Cluster 0 contains countries with relatively weaker socio-economic and healthcare indicators.

Examples include:

- **Afghanistan**, which has very high child mortality (90.2), extremely low income (1,610), very low GDP per capita (553), and low life expectancy (56.2), indicating severe development and healthcare challenges.
- **Angola** also belongs to this cluster due to extremely high child mortality (119.0) and relatively low life expectancy (60.1).

Although some countries such as **Argentina** and **Algeria** show moderate economic indicators, the cluster overall reflects countries facing economic and healthcare instability.

Cluster 1 — Developing / Transitional Economies

Cluster 1 represents countries with improving socio-economic conditions and moderate-to-high living standards.

Examples include:

- **Albania**, with moderate income (9,930), GDP per capita (4,090), and relatively strong life expectancy (76.3).
- **Bahrain** and **Belgium** demonstrate strong economic performance with high income levels (41,100), lower child mortality, and better healthcare outcomes.

This cluster captures developing and transitional economies with balanced economic growth and improving social indicators.

Cluster 2 — Developed / High Prosperity Countries

Cluster 2 contains economically advanced countries with strong healthcare systems and high quality of life.

Examples include:

- **Australia**, with very low child mortality (4.8), high income (41,400), high GDP per capita (51,900), and life expectancy of 82 years.
- **Austria** also demonstrates strong socio-economic performance with high GDP per capita (46,900) and high life expectancy (80.5).

Countries in this cluster generally exhibit:

- high income,
- strong healthcare indicators,
- lower mortality rates,
- and higher overall living standards.

This confirms that the clustering model successfully separated countries based on meaningful socio-economic development patterns.

Business Questions and Insights on Country Segmentation

1. If Standard Scaling was skipped, which features (e.g., gdp vs. life_expec) would unfairly dominate the clustering distance calculation?

Key Insight

K-Means and Hierarchical Clustering use Euclidean distance. Features with very large numerical ranges dominate the distance calculation.

Dominant Features

- gdp
- income

These features range in tens of thousands, while variables such as:

- life_expec
- total_fer
- child_mort

have much smaller ranges.

Business Impact

Without scaling:

- clustering would effectively group countries only by income/GDP.
- healthcare and demographic indicators would contribute very little.

Conclusion

Standard Scaling ensures:

- all features contribute equally,
 - clustering reflects complete socio-economic patterns.
-

2. Why might capping extreme high/low values (like max child_mort) be counterproductive when the goal is to identify countries in the most critical need of aid?

Key Insight

Extreme values in humanitarian datasets are often the actual signal — not noise.

Example

A country with:

- `child_mort = 180`

may represent a severe humanitarian crisis.

If capped to:

- `70`

the algorithm treats it similarly to moderately struggling countries.

Business Consequence

Aid targeting becomes diluted because:

- crisis countries lose their uniqueness,
- vulnerable nations become statistically hidden.

Conclusion

Outlier capping should be applied carefully in development economics datasets.

3. If K-Means yields a very high Silhouette Score, what does this imply about the separation and cohesion of the identified country segments?

Key Insight

A high Silhouette Score indicates:

- strong cluster cohesion,
- clear separation between clusters.

Interpretation

- Countries within a cluster are highly similar.
- Different clusters are meaningfully distinct.

Project Context

The obtained score (~0.2995) is moderate because:

- real-world development exists on a gradual spectrum,
- countries transition continuously between stages.

Conclusion

Moderate scores are expected in socio-economic clustering problems.

4. What unique policy insight does a Hierarchical Clustering Dendrogram offer that a K-Means scatter plot does not, regarding the relationships between clusters?

Key Insight

A dendrogram reveals:

- relationships between clusters,
- hierarchical similarity structure,
- natural cluster formation.

Unlike K-Means

K-Means only shows:

- final cluster assignments.

Dendrograms additionally show:

- how close clusters are to each other,
- gradual development transitions.

Business Value

Policy makers can:

- identify countries close to "graduating" into better segments,
 - design tiered intervention strategies.
-

5. Segment A has high income and high health spend. Segment B has low income and low health spend. What distinct policy goal should be set for Segment A vs. Segment B?

Segment A — High Income / High Health Spend

Policy Focus

- sustainability,
- healthcare equity,
- innovation,
- global partnerships.

Recommended Actions

- climate adaptation,
 - aging population healthcare,
 - global health collaboration.
-

Segment B — Low Income / Low Health Spend

Policy Focus

- basic survival infrastructure,
- healthcare access,
- poverty reduction.

Recommended Actions

- maternal healthcare,
 - sanitation infrastructure,
 - food security,
 - education access.
-

6. What is the business purpose of identifying the country closest to a cluster's centroid (the "representative country") for policy communication?

Key Insight

Cluster centroids are mathematical abstractions.

Representative countries:

- make clusters understandable,
- connect analysis to real-world policy discussions.

Business Benefits

- easier stakeholder communication,
- pilot program selection,
- simplified benchmarking.

Technical Method

Representative countries are identified using:

- Euclidean distance (`cdist`)
 - minimum distance to cluster centroid.
-

7. A cluster profile shows high average exports and imports but low gdpp. What is the immediate, tailored policy recommendation for this "Mid-Income Traders" segment?

Cluster Characteristics

- high exports,
- high imports,
- low GDP per capita.

Structural Problem

These countries often:

- export raw materials,
- import finished goods,
- capture limited economic value.

Recommended Policies

- invest in manufacturing,
- strengthen domestic value chains,
- improve logistics infrastructure,
- reduce trade friction costs.

Conclusion

The goal is:

- move countries higher in the global value chain,
- increase domestic economic retention.

Executive Summary

This project applied unsupervised machine learning techniques to identify hidden country segments using socio-economic indicators.

Key findings:

- PCA effectively reduced dimensionality
- K-Means produced well-separated clusters
- Hierarchical clustering revealed relationships between country groups
- DBSCAN identified vulnerable outlier nations

The analysis identified:

1. Developed economies
2. Emerging trade-focused economies
3. High-risk aid-dependent nations

These insights enable:

- targeted development programs
- efficient aid allocation
- evidence-based policymaking

Final Recommendation Summary

Cluster	Characteristics	Policy Recommendation
Underdeveloped	High child mortality, low GDP	Emergency healthcare and infrastructure aid
Developing	Moderate economic growth	Education and industrial investment
Developed	High income and healthcare	Sustainability and global partnership initiatives

Save Outputs

Final outputs are saved for:

- reporting
- dashboards
- future analysis

```
# Save clustered dataset
df.to_csv(
    "clustered_countries.csv",
    index=False
)

# Save evaluation metrics
evaluation_results.to_csv(
    "clustering_evaluation.csv",
    index=False
)
```

Reproducibility

- Random seed was fixed using RANDOM_STATE.
- All preprocessing, PCA, clustering, and evaluation steps are fully reproducible.
- The notebook can be executed sequentially from top to bottom without modification.

Limitations

- Country development exists on a continuous spectrum; clusters may overlap.
- DBSCAN performance was sensitive to eps parameter selection.
- Socio-economic indicators may change over time and require periodic updates.
- Clustering results depend on selected features and preprocessing decisions.

Future Improvements

- Incorporate time-series socio-economic data.
- Explore Gaussian Mixture Models and Spectral Clustering.
- Integrate geopolitical and environmental indicators.
- Build interactive dashboards for policy analysis.

Final Conclusion

The clustering analysis successfully identified meaningful socio-economic country segments using unsupervised learning techniques.

The results demonstrate that:

- underdeveloped countries require foundational healthcare and infrastructure support,
- developing countries benefit from industrial and educational investment,
- developed countries should focus on sustainability and global partnerships.

The segmentation framework provides data-driven support for global development organizations to allocate resources more strategically and maximize humanitarian impact.